

City Scan Technical Reference

World Bank - City Resilience Program

2026-05-05

Table of contents

Welcome	7
In this technical reference	7
About City Resilience Program (CRP)	7
 I City Scan Overview	 8
1 What is City Scan?	9
1.1 What can the City Scan tell me?	10
1.2 Guiding questions to keep in mind	10
1.3 Why invest in urban resilience	10
 2 City Scan Examples	 11
2.1 Example 1: Standalone Images - Dakar, Senegal	11
2.2 Example 2: Comprehensive Analytical PDF Report - Bitung, Indonesia	12
2.3 Example 3: Interactive HTML website - City, Country	12
 II Standard Workflow	 13
3 General Workflow	14
3.1 Part 1. Basic Setup	14
3.2 Part 2. Configuring Inputs	15
3.3 Part 3. Running a Scan	15
 4 Setup Instructions	 17
4.1 Step 1 — VS Code (Recommended)	17
4.2 Step 2 — Git	18
4.3 Step 3 — Clone the Repository	18
4.4 Step 4 — Python Environment	19
4.4.1 Option A: Conda (Recommended — Mac, Windows, Linux)	19
4.4.2 Option B: Bootstrap Script (Mac and Linux only)	20
4.5 Step 5 — R	21
4.6 Step 6 — Quarto	21
4.7 Step 7 — Google Cloud (gcloud)	22
4.7.1 7-1. Install gcloud CLI	22

4.7.2	7-2. Authenticate	22
4.7.3	7-3. Set GEE Project (Optional)	22
4.7.4	7-4. Verify Cloud Access	23
4.7.5	How Authentication Works in City Scan	23
4.7.6	Granting Access to a New Team Member	23
4.7.7	Troubleshooting	24
4.8	Step 8 — Verify Setup	24
5	Inputs Configuration	25
5.1	city_inputs.yml	25
5.1.1	Identification	25
5.1.2	Time ranges	26
5.1.3	Accessibility (osm_query + isochrone)	26
5.1.4	Flood (flood)	27
5.1.5	Benchmarks (benchmark_*, bm_cities_*, nearby_countries)	27
5.2	multi_inputs.yml	28
5.2.1	Mode A: per-city AOI folders	28
5.2.2	Mode B: multipolygon file	28
5.2.3	Mode B + filter	28
5.2.4	Shared settings	29
5.3	menu.yml	29
6	Running a Scan	31
6.1	Running Tasks	31
6.1.1	Running All Tasks	32
6.1.2	Running Individual or Multiple Tasks	33
6.1.3	Other Useful Flags	33
6.2	Multi-analysis	33
6.3	Rendering Maps and Charts	34
6.3.1	Maps	34
6.3.2	Charts	34
6.3.3	Scan Calculations Reference Sheet	34
III	Advanced Workflow	35
7	Running Multiple Cities	36
7.1	Option A: Separate AOI Folders	36
7.1.1	Step 1. Create inputs/multi_inputs.yml	36
7.1.2	Step 2. Organize AOI files by city	37
7.1.3	Step 3. Edit inputs/menu.yml	38
7.2	Option B: Multipolygon File	38
7.2.1	Step 1. Create inputs/multi_inputs.yml	38

7.2.2	Step 2. Edit <code>inputs/menu.yml</code>	39
7.3	Running	39
7.4	What Happens Under the Hood	40
7.5	Output Structure	40
8	Customizing Maps and Charts	41
8.1	Part 1: Maps	41
8.1.1	How map rendering works	41
8.1.2	Changing how a layer looks — <code>source/layers.yml</code>	41
8.1.3	Changing which layers a task renders — <code>tasks/{task}/maps.yml</code>	43
8.1.4	Adding a custom map script	44
8.2	Part 2: Charts	45
8.2.1	How chart rendering works	45
8.2.2	File structure	46
8.2.3	Anatomy of a chart document	46
8.2.4	Objects available in chart documents	48
8.2.5	Adding a new section to an existing chart	49
8.2.6	Creating a chart for a new task	49
IV	Tasks & Data Layers	50
9	Tasks & Structure Overview	51
9.1	What Is a Task?	51
9.2	Task Anatomy	51
9.2.1	Inputs, Processing, Outputs	51
9.2.2	Task Metadata	52
9.3	Task Categories	54
9.4	Execution Model	55
10	Data Layer References	56
10.1	Population	56
10.2	Land Use & Environment	56
10.3	Infrastructure	57
10.4	Risk & Hazard	59
10.5	Adding a Custom Data Layer	60
V	Understanding the Repo	61
11	Repository Structure	62
11.1	Top-level Layout	62
11.2	Core Package: <code>cityscan/</code>	62
11.2.1	Key Modules	63

11.3	Tasks Directory: <code>tasks/</code>	63
11.4	Inputs & Outputs	63
11.5	Tests	65
12	Task Dependency	66
12.1	Dependency Graph	68
12.2	How Dependencies Are Declared	69
12.3	Execution Order Example	69
12.4	Dependency Table	69
12.5	Handling Failures	70
12.6	Visualizing the DAG	70
VI	Case Studies & Galleries	71
13	Case Study: Standard Scan — Rabat, Morocco	72
13.1	City Context	72
13.2	AOI Definition	74
13.3	Running the Scan	75
13.4	Key Findings	75
13.4.1	Population Distribution	75
13.4.2	Flood Risk	77
13.4.3	Land Use	77
13.5	Outputs	77
14	Case Study: Multiple Scans — Philippines	78
14.1	Project Context	78
14.2	Cities Covered	78
14.3	Batch Configuration	78
14.4	Running the Batch	79
14.5	Key Findings Across Cities	81
14.5.1	Coastal Flood Risk	82
14.5.2	Urban Heat Islands	82
14.6	Lessons Learned	82
VII	Collaborating	83
15	Git Guide	84
15.1	Branching Strategy	84
15.2	Cloning the Repository	84
15.3	Commit Message Conventions	84
15.3.1	Examples	85
15.4	Pull Request Workflow	85

15.5 Keeping Your Branch Up to Date	87
16 Reporting Issues	88
16.1 Where to Report	88
16.2 Before You Open an Issue	88
16.3 Bug Reports	88
16.4 Feature Requests	89
16.5 Sharing Log Files	91
16.6 Issue Labels	91
17 Contributing	92
17.1 Types of Contributions	92
17.2 Development Setup	92
17.3 Code Style	92
17.4 Writing a New Task	93
17.5 Pull Request Checklist	95
17.6 Review Process	96
18 Sharing the Guide	97
18.1 Rendered Formats	97
18.2 Publishing to GitHub Pages	97
18.3 Publishing to Quarto Pub	98
18.4 Sharing the PDF	98
18.5 Embedding in Other Sites	99
18.6 Keeping the Guide Up to Date	99
19 Editing This Reference	101
19.1 Prerequisites	101
19.2 Repository Structure	101
19.3 Editing Workflow	102
19.3.1 1. Switch to <code>new_docs</code>	102
19.3.2 2. Edit <code>.qmd</code> files	102
19.3.3 3. Add a new page (optional)	102
19.3.4 4. Commit and push to <code>new_docs</code>	102
19.4 Publishing Workflow	103
19.5 Branch Overview	103
19.6 Quick Reference	104
References	105

Welcome

The City Scan is a diagnostic tool designed to equip cities with the early insights needed to address risk by:

- Providing a rapid, data-driven snapshot of a city's demographic, economic, climate, and risk landscape.
- Leveraging remote sensing and geospatial analysis of any city in the world, using global datasets.
- Revealing the interaction between the urban built and natural environments.

In this technical reference

This reference book is a structured technical documentation to use and operate City Scan as a diagnostic tool. Here, you will find documentations on City Scan overview, getting started, running a City Scan, tasks and data layers, repository structure, case studies & galleries, as well as contributing guides.

About City Resilience Program (CRP)

The City Resilience Program is housed within the Global Facility for Disaster Reduction and Recovery (GFDRR) at the World Bank. CRP is a global initiative supported by the Swiss State Secretariat for Economic Affairs, the Austrian Federal Ministry of Finance, the Italian Ministry of Foreign Affairs, the Italian Agency for Development Cooperation, and the Gates Foundation.

To learn more about CRP and GFDRR visit <https://www.gfdr.org/en/crp>.

Part I

City Scan Overview

1 What is City Scan?



Figure 1.1: City Scan Concept

A City Scan is a rapid geospatial assessment of a city’s demographic, socioeconomic, climate, and risk conditions. It is a collection of maps and charts made from global and publicly available datasets that provide quick high-level insights into resilience-related topics for a city. The output is a package of maps, data visualizations and insights that integrate features of both the built and natural environments. These scans are especially useful for grounding early-stage conversations in geospatial data.

1.1 What can the City Scan tell me?

This City Scan is intended to support operational teams in building dialogue around a city’s most pressing resilience challenges. It is designed as a conversation starter rather than a specific decision making tool. Thinking spatially about how urbanization affects the resilience of various urban forces, networks, and people helps equip officials to develop risk informed investment proposals, identify opportunities and barriers to unlocking private capital, and prioritize and coordinate future investments.

1.2 Guiding questions to keep in mind

- How might investment priorities differ in different parts of the city based on this data?
- What spatial relationships are surprising?
- What questions does this spatial pattern spur?
- Where else have you worked that is reminiscent of these findings?
- What might be applicable from there? What might not be?

1.3 Why invest in urban resilience

Investing in urban resilience boosts long-term economic, social, and physical sustainability by safeguarding the gains from current developments for future generations. Greater urban resilience builds the capacity of local governments to help households, communities, and enterprises manage and avoid shocks and stresses, for example by damage-proofing infrastructure before a disaster or maintaining critical services afterward. This approach represents a significant shift from prior development trends, which concentrated investments on post-disaster reconstruction and recovery.

Note

For a hands-on introduction, see [Running Your First City Scan](#).

2 City Scan Examples

City Scan output is a collection of maps and charts made from global and publicly available datasets that provide quick high-level insights into resilience-related topics for a city. The output is a package of maps, data visualizations and insights that integrate features of both the built and natural environments. It is available as standalone images, compiled PDF, and interactive Quarto HTML.

2.1 Example 1: Standalone Images - Dakar, Senegal

This format is useful if you'd like to combine City Scan outputs with your own analysis. See examples from our analysis for Dakar, Senegal

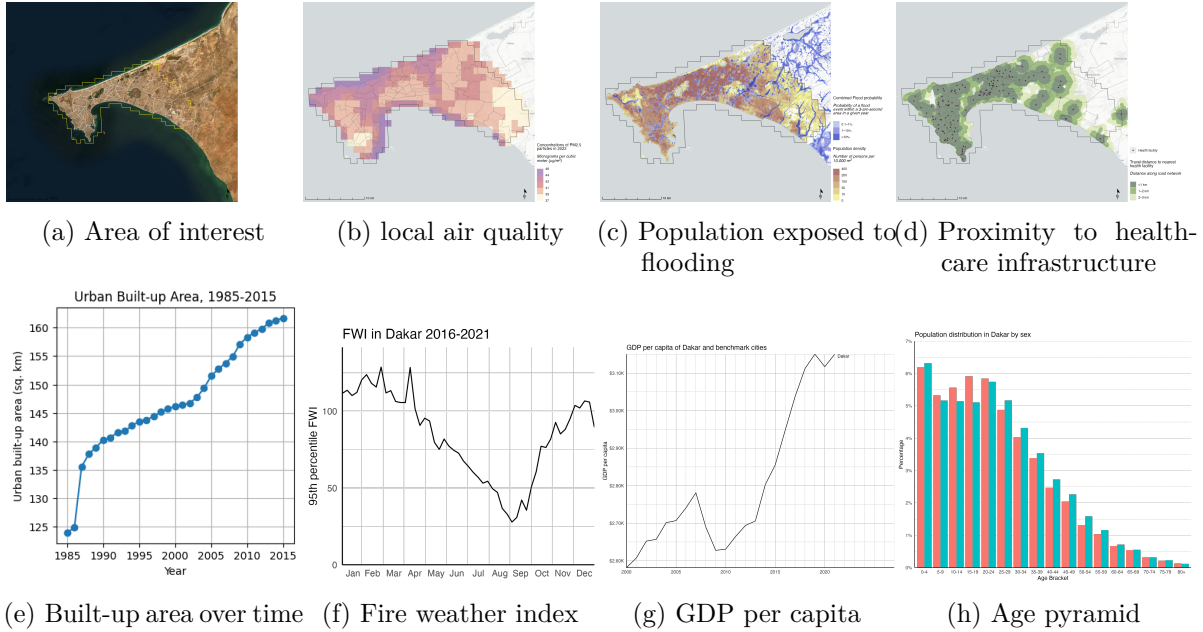


Figure 2.1: Examples of maps & charts generated by City Scan

2.2 Example 2: Comprehensive Analytical PDF Report - Bitung, Indonesia

The generated maps and charts are compiled according to relevant themes, along with data interpretation and considerations. See examples below from our analysis in Bitung, Indonesia.

Note

The final PDF output is compiled and designed via Adobe InDesign and not yet automatically generated by the code. Automated PDF generation will be part of future updates.

2.3 Example 3: Interactive HTML website - City, Country

Nemo enim ipsam voluptatem quia voluptas sit aspernatur aut odit aut fugit, sed quia consequuntur magni dolores eos qui ratione voluptatem sequi nesciunt.

Tip

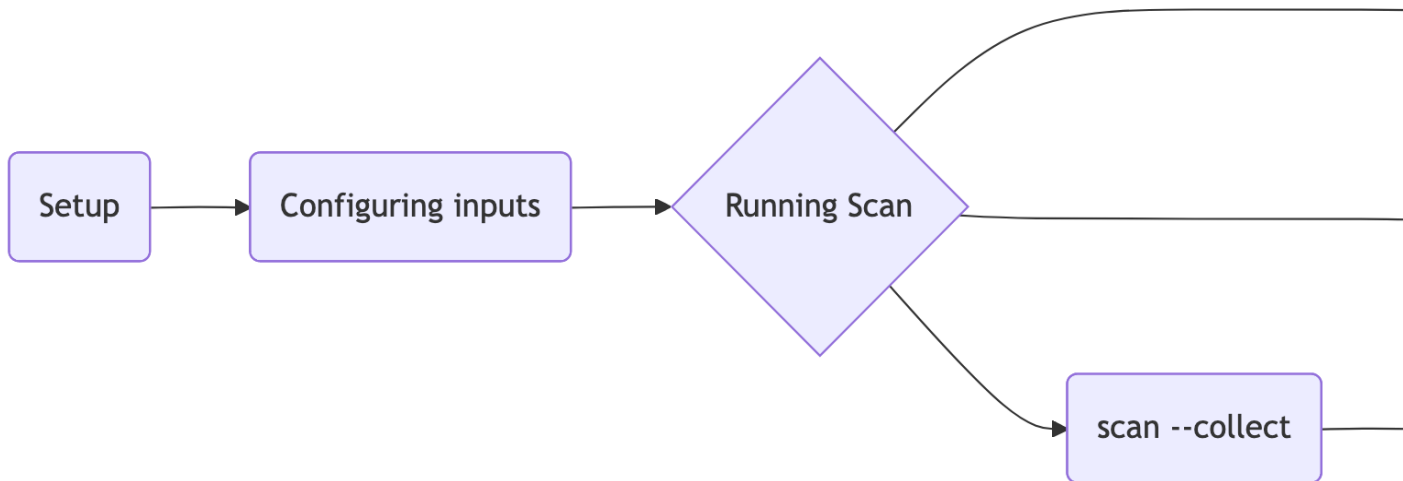
Full case studies for Rabat and the Philippines are in [Part 6. Case Studies & Galleries](#).

Part II

Standard Workflow

3 General Workflow

This chapter provides a three-part walkthrough of the City Scan repo for new users. The diagram below illustrates the general workflow of performing a standard City Scan.



3.1 Part 1. Basic Setup

i Note

See [Part 4. Getting Started/ Setup Instructions](#) for full details of prerequisites and installations

City Scan uses multiple programming languages and software to run, including Git, Python, R, Quarto, and Google Cloud. The steps required for setups are below:

- Clone the repo <https://github.com/cityresilience/city-scan>
- Create conda env + activate cityscan
- Installing City Scan as Python Package via `pip install -e .` for the scan CLI

- GCS and GEE authentication via `gcloud auth application-default login`
- Install Quarto
- Run `scan --check` to verify everything passes

3.2 Part 2. Configuring Inputs

Inputs folder is where you can specify your area of interests (AOI), types of analysis to conduct, and certain global analysis parameters.

- Drop AOI shapefile into `inputs/AOI/`
- Edit `inputs/city_inputs.yml`
- Edit `inputs/menu.yml`

3.3 Part 3. Running a Scan

When running a scan, we are essentially going through several phases. The general flow is collect → analyze → multianalysis → render.

1. **Collect** stage is retrieving specified datasets as indicated in inputs configuration from various publicly available sources for the specified AOI. As an output, you will retrieve TIFFs, GeoJSONs, or Geopackages in your output folder.

To conduct **collect** stage, run in your terminal

```
scan --collect
```

2. **Analyze** stage is conducting various analysis on a single task, such as seeing the statistical distribution, applying threshold, extracting median/mean values, etc. As an output, you will retrieve various CSVs in your output folder.

To conduct **analyze** stage, run in your terminal

```
scan --analyze
```

3. Alternatively, you can **batch data collection and data analysis together by looping over tasks** by running:

```
scan --all
```

Or, you can leverage your computer memory and **parallelizing data collection and data analysis together per tasks** by running:

```
scan --all --parallel
```

 Warning

[**Internal Note**] Perhaps in the future, `scan --all` and `scan --all --parallel` should be reserved to orchestrate the whole process, including multianalysis, render, and scan-calculations

4. **Multianalysis** stage is performing cross-task analysis after single tasks analysis have been executed. For example, this stage executes analysis of flood-exposed population, which requires both flood analysis and population analysis to have been conducted. To do this, run in your terminal:

```
scan --multianalysis
```

5. **Render** stage is generating various visualizations as results of previous stages. This includes rendering maps and charts as PNGs or HTMLs.

```
# Static maps - applies styling from source/layers.yml, writes PNGs to 03-render-output/  
scan --render maps
```

```
# Charts for a specific task - writes HTML to tasks/{name}/charts/index.html  
scan --render charts elevation
```

To generate a compiled interactive quarto document and HTML of all tasks, run:

```
scan --render scan-calculations
```


4 Setup Instructions

City Scan is multilingual: Python handles data collection and analysis, R generates maps and charts, and Quarto compiles reports and web visualizations. The `scan` CLI ties it all together.

Work through the sections below in order. The checklist from [General Workflow](#):

- ☐ Install prerequisites (VS Code, Git, Python, R, Quarto, gcloud)
- ☐ Clone the repository
- ☐ Create and activate the `cityscan` conda environment
- ☐ Install City Scan as a Python package via `pip install -e .`
- ☐ Authenticate with GCS and GEE
- ☐ Run `scan --check` to verify everything passes

4.1 Step 1 — VS Code (Recommended)

VS Code is a free code editor with an integrated terminal and Git UI. It handles Python, R, and Quarto in the same window, which makes the City Scan workflow much smoother.

Install from code.visualstudio.com.

Recommended extensions — install from the Extensions panel (`Ctrl+Shift+X` / `X`):

Extension	Purpose
Python	Python language support and environment selection
R	R language support
Quarto	Quarto rendering and preview
Rainbow CSV	Color-coded columns for CSVs under <code>02-process-output/tabular/</code>
Color Highlight	Inline color swatches for hex codes in <code>source/layers.yml</code>
Geo Data Viewer	Preview <code>.geojson</code> and <code>.shp</code> files on a map

Extension	Purpose
GeoTIFF Viewer	Preview <code>.tif</code> rasters directly in the editor

The following steps require you to use the terminal. In VS Code topbar, go to **Terminal** > **New Terminal**

4.2 Step 2 — Git

Git is a version control system that tracks changes and enables collaboration through GitHub. It is required to clone the repository and stay up to date with changes.

Git is pre-installed on most macOS and Linux systems. Verify with:

```
git --version
```

If not installed, follow the [GitHub Git installation guide](#). Windows users can use [Git for Windows](#).

4.3 Step 3 — Clone the Repository

```
git clone https://github.com/cityresilience/city-scan
cd city-scan

# switch to unified branch
git switch unified
```

All subsequent steps assume you are inside the `city-scan/` directory.

4.4 Step 4 — Python Environment

City Scan requires Python 3.11. Choose one of the two options below.

4.4.1 Option A: Conda (Recommended — Mac, Windows, Linux)

4.4.1.1 4A-1. Install Anaconda

Download and install from anaconda.com/download. Accept the default settings and restart your terminal after installation.

Verify:

```
conda --version
```

💡 conda: command not found after install?

Conda was installed but not initialized for your shell. Run the appropriate command once:

```
conda init zsh      # macOS / Linux with zsh (default on modern macOS)
conda init bash     # Linux / older macOS / WSL
conda init powershell # Windows PowerShell
```

Close and reopen the terminal. You should see (base) in your prompt.

4.4.1.2 4A-2. Create and Activate the Environment

```
conda create -n cityscan python=3.11.8 -y
conda activate cityscan
```

4.4.1.3 4A-3. Install City Scan & requirements

```
python -m pip install --upgrade pip
pip install -e .
pip install -r requirements.txt
```

4.4.1.4 4A-4. Register Jupyter Kernel (Optional)

Only needed if you plan to use Jupyter notebooks:

```
python -m ipykernel install --user --name cityscan --display-name "Cityscan"
```

4.4.2 Option B: Bootstrap Script (Mac and Linux only)

Use this option if you prefer automated system setup or do not have Python installed.

4.4.2.1 4B-1. Run the Bootstrap Script

This installs Python 3.11.8 via pyenv and prepares your system:

```
bash bootstrap.sh
```

Note

Review `bootstrap.sh` before running. It installs system dependencies, pyenv, and Python 3.11.8, and updates your shell configuration. If pyenv is being installed for the first time, restart your terminal and run the script again.

4.4.2.2 4B-2. Run the Setup Script

Creates the virtual environment, installs dependencies, and registers the Jupyter kernel:

```
bash orchestrator.sh
```

When successful, you will see:

- Python 3.11.8 ready via pyenv
- A new virtual environment created
- A Jupyter kernel registered: “Cityscan (Python 3.11)”

To activate the environment manually later:

```
source venv/bin/activate
```

4.5 Step 5 — R

R is used for maps, charts, and the scan-calculations report, and also for several data collection tasks (earthquake, Oxford, cyclones, flood events, GDP gridded, coastal erosion, sea level rise).

Download the **latest stable R release** (4.5.x or newer) from cran.r-project.org. The repo depends on a current `ggplot2`, `terra`, `sf`, and `tidyterra` stack that will not install cleanly on older R versions.

Verify:

```
R --version
```

R in VS Code: Install the [R extension](#) and, optionally, [radian](#) for a richer R console experience.

Note

RStudio works fine for R-specific work and can be downloaded from posit.co. For the full City Scan workflow (Python + R + terminal), VS Code is recommended.

4.6 Step 6 — Quarto

Quarto is the publishing system used to compile City Scan reports and the web visualization. It supports Python, R, JavaScript, and Julia in the same document.

Install from quarto.org/docs/get-started.

Verify:

```
quarto check
```

4.7 Step 7 — Google Cloud (gcloud)

City Scan uses Google Cloud for two services:

- **Google Earth Engine (GEE)** — satellite imagery and geospatial analysis
- **Google Cloud Storage (GCS)** — private data buckets (`city-scan-global-data`, `city-scan-gcs-data`)

If you do not yet have access to these resources, contact the project lead to be added to the Google Cloud project.

4.7.1 7-1. Install gcloud CLI

Download and install from cloud.google.com/sdk/docs/install. After installation, initialize the SDK:

```
gcloud init
```

Verify:

```
gcloud --version
```

4.7.2 7-2. Authenticate

```
gcloud auth application-default login
```

This opens a browser window to complete the OAuth flow. Credentials are stored locally and picked up automatically by City Scan:

- **macOS / Linux:** `~/.config/gcloud/application_default_credentials.json`
- **Windows:** `%APPDATA%\gcloud\application_default_credentials.json`

4.7.3 7-3. Set GEE Project (Optional)

The default GEE project is `city-scan-gee-test`. To override:

```
export GEE_PROJECT=your-project-id
```

4.7.4 7-4. Verify Cloud Access

```
# Check GEE
python -c "import ee; ee.Initialize(); print('GEE OK!')"

# Check GCS
gcloud storage ls gs://city-scan-global-public/ --limit=1
```

4.7.5 How Authentication Works in City Scan

Authentication is managed in `core/config/`:

File	Role
<code>auth.py</code>	Initializes GEE and GCS credentials
<code>scan.py</code>	Loads AOI, city config, and creates output folders
<code>paths.py</code>	Auto-detects input/output directories
<code>gdal_auth.py</code>	Configures GDAL for access to private GCS buckets via <code>/vsigs/</code>

Tasks that require GEE (forest, landcover, LST, NDVI, nightlight) or private GCS (WSF, fathom, landcover burn, basic info) are authenticated automatically. If authentication fails, those tasks are skipped with a warning.

Warning

Never commit credentials or JSON key files to the repository. Treat them like passwords.

4.7.6 Granting Access to a New Team Member

1. Go to [IAM & Admin](#) in the Google Cloud Console
2. Click **Grant access**
3. Enter the new user's Gmail address under **New principals**
4. Assign the role **Editor**
5. Click **Save**

The new user then runs the following to authenticate locally:

```
gcloud auth login
gcloud auth application-default login
```

4.7.7 Troubleshooting

Symptom	Fix
GEE auth fails	Re-run <code>gcloud auth application-default login</code>
GCS 403 / 404 error	Confirm the account has the Storage Object Viewer role on the relevant bucket
“GCS credentials not found”	Check that <code>~/.config/gcloud/application_default_credentials.json</code> exists

4.8 Step 8 — Verify Setup

Run the built-in check from the repo root:

```
scan --check
```

This verifies that Python dependencies, R packages, Quarto, and cloud credentials are all in place. Fix any reported issues before running a scan.

5 Inputs Configuration

The `inputs/` folder controls everything about a scan: which city, which AOI, which tasks run, and the parameters for each task.

Three YAML files drive the pipeline:

File	Purpose
<code>city_inputs.yml</code>	Single-city config: city name, AOI, year ranges, flood/accessibility/benchmark params
<code>multi_inputs.yml</code>	Multi-city batch config: list of cities (or a multipolygon file) + shared settings
<code>menu.yml</code>	Per-task on/off toggles

Templates live in `templates/`. Copy them to `inputs/` and edit before running.

5.1 `city_inputs.yml`

5.1.1 Identification

Key	Type	Description
<code>city_name</code>	string	Display name. Spaces and accents OK (<code>Lobito Corridor</code> , <code>Lüderitz</code>). Used for outputs, labels, and slugified <code>scan_id</code> .

Key	Type	Description
AOI_shp_name	string (optional)	Stem of the AOI shapefile in inputs/AOI/ (no extension). If omitted, AOI is auto-detected from UCDB / OSM / 5km buffer around the city centroid.
prev_run_date	string YYYY-MM (optional)	Reuse a previous scan folder. Default: null (creates fresh mnt/{YYYY-MM}-{country}-{city}/).

5.1.2 Time ranges

Key	Type	Description
first_year	int	Start year for LST and time-series tasks. Sentinel-2 / Landsat data available from 2013-03-18.
last_year	int	End year for the same.
fwi_first_year	int	Start year for Fire Weather Index.
fwi_last_year	int	End year for FWI.
demographics_year	int (optional)	WorldPop R2025A year (2015–2030). Default: current year.

5.1.3 Accessibility (osm_query + isochrone)

OSM POI categories to fetch and walking-distance buffers for each.

```
osm_query:
  schools:
    amenity: [school, kindergarten, university, college]
  health:
    amenity: [clinic, hospital, health]
  police:
    amenity: [police]
  fire:
```

```

    amenity: [fire_station]

isochrone: # meters
    schools: [800, 1600, 2400]
    health: [1000, 2000, 3000]

accessibility_buffer: 5 # % of AOI bounds to buffer for capturing amenities just outside the

```

5.1.4 Flood (flood)

Drives the Fathom flood analysis.

```

flood:
    threshold: 15 # cm - minimum depth to count as flooded
    year: [2020, 2050, 2100]
    ssp: [2, 5] # available: 1, 2, 3, 5
    return_period: [10, 100, 1000] # available: 5, 10, 20, 50, 100, 200, 500, 1000

```

5.1.5 Benchmarks (benchmark_*, bm_cities_*, nearby_countries)

Controls which cities to benchmark against for population/Oxford comparisons.

Key	Values	Description
benchmark_mode	sibling / oxford / auto (default)	sibling = only sibling scans; oxford = only Oxford auto-detect; auto = both.
bm_cities_manual	list of city names	Explicit benchmark cities. Resolved as sibling → Oxford → backup, in that order.
benchmark_backup	citypopulation / worldpop_ucdb / null	Fallback for manual cities not found as sibling or Oxford.
nearby_countries	pipe-separated country names	Filter for Oxford auto-detect (only relevant when mode includes Oxford).

5.2 multi_inputs.yml

Same fields as `city_inputs.yml` plus the `cities/multipolygon` list. Two modes — use one (or combine, see below).

5.2.1 Mode A: per-city AOI folders

Each city has its own `inputs/AOI/{city_name}/` folder with a `.shp`. Wards auto-detected from `inputs/AOI/{city_name}_wards/` if present.

```
cities:
- city_name: Windhoek
- city_name: Swakopmund
- city_name: Walvis Bay
- city_name: Lüderitz
- city_name: Aus
```

Per-city overrides supported:

```
cities:
- city_name: Windhoek
  first_year: 2010      # overrides the shared first_year for this city only
- city_name: Aus
```

5.2.2 Mode B: multipolygon file

A single vector file (GPKG, SHP, GeoJSON) where each row is a city. AOIs are auto-extracted to `inputs/AOI/{city_slug}/`.

```
multipolygon:
  file: inputs/AOI/lobito_corridor_cities.gpkg
  name_column: GC_UCN_MAI_2025
```

5.2.3 Mode B + filter

Combine `multipolygon:` with a `cities:` list to extract all AOIs but only run a subset.

```
multipolygon:
  file: inputs/AOI/lobito_corridor_cities.gpkg
  name_column: GC_UCN_MAI_2025

cities:
  - city_name: Sakania
  - city_name: Lukuni
```

If a name in `cities:` isn't found in the multipolygon file, the run errors out.

5.2.4 Shared settings

All other `city_inputs.yml` keys (flood, accessibility, FWI, year ranges, `benchmark_*`) work identically here and apply to every city in the list.

5.3 menu.yml

Per-task on/off. Set `True` to run, `False` to skip. Tasks not listed default to `False`.

```
accessibility: True
air_quality: True
basic_info: True
buildings: True
burned_area: True
coastal_erosion: True
cyclones: True
demographics: True
earthquake: True
elevation: True
flood_coastal: True
flood_fluvial: True
flood_pluvial: True
flood_comb: True
flood_events: True
forest: True
fwi: True
ghs_builtup: True
```

```
ghs_population: True
green: True
gdp_gridded: True
gdp_sectoral: True
groundwater: True
landcover: True
landcover_burn: True
landslide: True
liquefaction: True
lst_summer: True
lst_winter: True
ndmi: True
nightlight: True
oxford: True
population: True
rwi: True
sea_level_rise: True
seismic_hazard: True
slope: True
solar: True
water_risk: True
wsf: True

# Compilation
scan_calculations: True
```

Notes: - `population` = WorldPop. To use GHSL instead, set `population: False` and `ghs_population: True`. - `slope` is derived from `elevation`. If you toggle `slope: True`, also enable `elevation: True` (unless the elevation raster is already on GCS). - `scan_calculations: True` triggers the combined HTML report after all tasks finish. The folder is auto-copied to the city directory.

See `templates/menu.yml` and `tasks/` for the canonical task list.

6 Running a Scan

6.1 Running Tasks

i Note

Activate the `cityscan` Python environment before running any `scan` commands.

Tasks are run from the `city-scan` directory or from within `mnt/scan-id` using the general form:

```
scan {TASKNAME} {FLAG}
```

i Note

What is a scan-id? It is an automatically generated string with the format `yyyy-mm-country-city` (e.g. `2026-02-malta-malta`). It is good practice to always specify the scan-id explicitly when running `scan` commands.

Each task is divided into steps defined by `{FLAG}`:

Flag	What it does
<code>--collect</code>	Runs data collection; saves outputs to <code>02-process-outputs/spatial/</code> or <code>/tabular/</code>
<code>--analyze</code>	Runs data analysis; creates output CSVs in <code>02-process-outputs/tabular/</code>
<code>--all</code>	Runs both <code>--collect</code> and <code>--analyze</code> for all tasks enabled in <code>menu.yml</code>
<code>--multianalysis</code>	Runs cross-task analysis (currently flood exposure)

6.1.1 Running All Tasks

To run all tasks enabled in `menu.yml`:

```
scan --all
```

This opens a Terminal UI (TUI) showing per-task progress and logs.

Behavior depends on where you run it from:

- **From `city-scan/` root** — initializes `mnt/scan-id` and copies all necessary folders
- **From inside `mnt/scan-id`** — writes outputs into the local `02-process-outputs/`

To target an existing scan by ID:

```
scan --all --parallel --scan-id 2026-02-malta-malta
```

You will see something like this in your terminal when running in parallel:

```
manila manila 0 4 32 0
  accessibility      collect  INFO | tasks.accessibility.collection | Downloading all 3
  air_quality         collect  INFO | tasks.air_quality.collection | AOI CRS: EPSG:432 3
  basic_info          collect  city-scan-gee-test 37s
  buildings           waiting  (osm)
  burned_area         collect  INFO | tasks.burned_area.collection | Starting burned a 3
  coastal_erosion     waiting  (default)
  cyclones            waiting  (default)
  demographics        waiting  (default)
  earthquake          waiting  (default)
  elevation           waiting  (gcs)
  wsf                 waiting  (gcs)
  oxford              waiting  (default)
  worldpop            waiting  (default)
  fathom              waiting  (gcs)
  flood_events        waiting  (default)
  forest              waiting  (gee)
  fwi                 waiting  (default)
  gdp_gridded         waiting  (default)
  gdp_sectoral        waiting  (default)
  ghs_builtup         waiting  (default)
[↑↓] select  [Enter] log  [q] quit
[↑↓] scroll   [Esc] back  [q] quit
```


6.1.2 Running Individual or Multiple Tasks

```
# Single task
scan wsf

# Multiple tasks
scan wsf population forest
```

To run a specific step for one or more tasks:

```
# Collect only
scan wsf --collect

# Analyze only, multiple tasks
scan wsf population green --analyze
```

6.1.3 Other Useful Flags

```
# List all available tasks
scan --list

# Re-run using the city's existing code (no resync from root)
scan --all -k --scan-id 2026-02-malta-malta
```

6.2 Multi-analysis

Some tasks have cross-task analysis scripts — for example, flood-exposure calculations that combine flood and WSF outputs. Run these after all individual tasks have completed:

```
scan --all --multianalysis
```

6.3 Rendering Maps and Charts

After data collection and analysis, generate visualizations using the `--render` flag.

6.3.1 Maps

Applies styling from `source/layers.yml` and writes PNGs to `03-render-output/maps/`:

```
scan --render maps --scan-id 2026-02-malta-malta
```

Maps read spatial data from `02-process-output/spatial/`.

6.3.2 Charts

Writes an HTML file to `tasks/{name}/charts/index.html`:

```
scan --render charts elevation --scan-id 2026-02-malta-malta
```

Charts read CSVs from `02-process-output/tabular/`.

6.3.3 Scan Calculations Reference Sheet

The `scan-calculations` render reads `sections.yml` to determine section order, then assembles all per-task `charts/index.qmd` files into a single combined HTML report:

```
scan --render scan-calculations --scan-id 2026-02-malta-malta
```

The compiled report is saved to `scan-calculations/scan-calculations.html`.

i Note

If `scan_calculations: True` is set in `menu.yml`, the `scan-calculations` folder is automatically copied to the city folder when running `scan --all`.
See [scan-calculations/README.md](#) for details.

Part III

Advanced Workflow

7 Running Multiple Cities

While a single-city run processes one city at a time, the `--multicity` flag lets you queue a list of cities and run them sequentially from a shared configuration file. Setup is similar, but inputs are organized per city rather than in a flat folder.

There are two ways to provide city boundaries for a multi-city run. Both use the same `inputs/multi_inputs.yml` config file and the same `scan --all --multicity` command.

7.1 Option A: Separate AOI Folders

Best when you already have individual shapefiles per city.

7.1.1 Step 1. Create `inputs/multi_inputs.yml`

This file follows the same structure as `city_inputs.yml`, with a `cities:` list added at the top.

```
cities:
  - city_name: Windhoek
  - city_name: Swakopmund
  - city_name: Walvis Bay
  - city_name: Lüderitz
  - city_name: Aus

# Shared settings - same fields as city_inputs.yml
first_year: 2015
last_year: 2026

flood:
  threshold: 15
  year: [2020]
  return_period: [10, 100, 1000]

# ... accessibility, FWI, etc.
```

All settings outside the `cities:` list are shared across every city. You can override any shared setting for a specific city by adding it under that city's entry:

```
cities:
- city_name: Windhoek
  first_year: 2010    # overrides the shared first_year for Windhoek only
- city_name: Aus
```

7.1.2 Step 2. Organize AOI files by city

Each city needs its own subfolder under `inputs/AOI/`. Place the shapefile and its companion files (`.dbf`, `.prj`, `.shx`) inside that subfolder:

```
inputs/AOI/
  Windhoek/
    Windhoek_geoboundaries.shp
    Windhoek_geoboundaries.dbf
    ...
  Swakopmund/
    Swakopmund.shp
    ...
  Walvis_Bay/
    walvis_buffer_hull_1000m.shp
    ...
```

If a city has ward boundaries, place them in a sibling folder named `{city_name}_wards/`:

```
inputs/AOI/
  Windhoek/
    Windhoek_geoboundaries.shp
    ...
  Windhoek_wards/
    Windhoek_wards_geoboundaries.shp
    ...
```

Wards are auto-detected — no extra configuration needed.

7.1.3 Step 3. Edit `inputs/menu.yml`

The same `menu.yml` is used for all cities. Enable or disable tasks there as usual.

7.2 Option B: Multipolygon File

Best when all city boundaries are already in a single vector file (GPKG, SHP, or GeoJSON), with one row per city. AOI shapefiles are extracted automatically from the file into `inputs/AOI/{city_slug}/` — you don't need to prepare them manually.

7.2.1 Step 1. Create `inputs/multi_inputs.yml`

Instead of organizing AOIs into subfolders, point to the multipolygon file and specify which column holds the city names:

```
multipolygon:
  file: inputs/AOI/lobito_corridor_cities.gpkg
  name_column: GC_UCN_MAI_2025

# Shared settings - same as Option A
first_year: 2015
last_year: 2026

flood:
  threshold: 15
  year: [2020]
  return_period: [10, 100, 1000]
```

By default, all cities in the file are processed. To run only a subset, add a `cities:` list — all AOIs are still extracted, but only the listed cities are scanned:

```
multipolygon:
  file: inputs/AOI/lobito_corridor_cities.gpkg
  name_column: GC_UCN_MAI_2025

cities:
  - city_name: Sakania
  - city_name: Lukuni
```

i Note

If a city listed under `cities:` is not found in the multipolygon file, an error is raised.

7.2.2 Step 2. Edit `inputs/menu.yml`

Same as Option A — one shared `menu.yml` for all cities.

7.3 Running

```
scan --all --multiplicity
```

Cities are always processed sequentially. All other flags work as expected:

```
scan --all --multiplicity --parallel      # parallel tasks within each city
scan --all --multiplicity --collect       # collect step only, all cities
scan wsf fathom --multiplicity            # specific tasks for all cities
scan --all --multiplicity --upload        # upload to GCS after each task

# Render for all cities
scan --render maps --multiplicity
scan --render scan-calculations --multiplicity
```

i Note

`--parallel` applies to tasks *within* each city, not across cities. Cities themselves always run one at a time.

7.4 What Happens Under the Hood

For each city in the list, the scan runner:

1. Extracts the city's AOI — either from a `{city_name}/` subfolder in `inputs/AOI/` (Option A) or from the multipolygon file (Option B, which writes shapefiles to `inputs/AOI/{city_slug}/`)
 2. Auto-detects the `.shp` file inside that subfolder
 3. Generates a `city_inputs.yml` combining shared settings, any city-specific overrides, the detected `AOI_shp_name`, and a `bm_cities_manual` list populated with the other cities
 4. Writes that `city_inputs.yml` to `inputs/` and runs `scan` as a subprocess
 5. The normal single-city flow takes over: creates `mnt/{scan_id}/`, copies AOI and wards files flat into `01-user-input/`, and runs the enabled tasks
 6. Moves on to the next city
-

7.5 Output Structure

Each city produces its own self-contained folder under `mnt/`:

```
mnt/  
  2026-04-namibia-windhoek/  
  2026-04-namibia-swakopmund/  
  2026-04-namibia-walvis_bay/  
  2026-04-namibia-luderitz/  
  2026-04-namibia-aus/
```

The `bm_cities_manual` field links each city to its siblings, enabling cross-city population comparisons in the `worldpop` analysis.

8 Customizing Maps and Charts

City Scan produces two kinds of visual outputs: static map PNGs (one per data layer) and interactive HTML chart reports (one per task). Both are customizable without touching the core pipeline — maps through YAML config files, charts through Quarto documents.

8.1 Part 1: Maps

8.1.1 How map rendering works

Running `scan --render maps --scan-id {id}` executes `core/R/maps-static.R`, which reads two config files and renders one PNG per layer into `03-render-output/maps/`:

- **`source/layers.yml`** — the global style library. Each entry defines a *yaml_key* (e.g. `population`) with its title, palette, breaks, and other visual settings.
- **`tasks/{task}/maps.yml`** — per-task config. Declares which *yaml_keys* this task contributes, whether to aggregate raster pixels into hexbins, and whether custom R scripts are needed.

The renderer matches each *yaml_key* to a file in `02-process-output/spatial/` using the `fuzzy_string` pattern, then applies the style settings to produce the map.

8.1.2 Changing how a layer looks — `source/layers.yml`

This is the file to edit when you want to change colors, legend labels, title text, or binning for any existing layer.

Each entry is keyed by a short identifier (the *yaml_key*) that the rest of the system uses to refer to that layer:

```

population:
  fuzzy_string: (?<!dense_)population.tif$
  title: 'Population density'
  subtitle: 'Number of persons per 10,000 m<sup>2</sup>'
  palette: ['#ECEB72', '#8E3933']
  bins: 5
  basemap: 'satellite'
  group_id: Population density

```

The fields you are most likely to modify:

Field	What it controls
<code>title</code>	Main legend header
<code>subtitle</code>	Secondary legend text (supports HTML)
<code>palette</code>	List of hex colors from low to high value
<code>bins</code>	Number of equal-interval bins (alternative to explicit <code>breaks</code>)
<code>breaks</code>	Explicit cut points including limits; for categorical maps, the factor levels
<code>labels</code>	Human-readable label for each bin (must match the number of intervals)
<code>oob</code>	Out-of-bounds handling — <code>sensor</code> clips values outside the range
<code>suffix</code>	Unit string appended to legend values (e.g. °C, %)
<code>factor: TRUE</code>	Forces categorical (discrete) rendering

Example: Change the flood return period labels from short codes to plain English:

```

fluvial:
  title: 'Fluvial flood return period'
  labels: ['1-in-1,000 year', '1-in-100 year', '1-in-10 year']
  palette: ['#A7BCEE', '#1B35C4']
  breaks: [0, .11, 1.1, 10.1]

```

i Note

The `fuzzy_string` field is a regex used to find the data file in `02-process-output/spatial/`. Only change it if the task's output filename has changed.

8.1.3 Changing which layers a task renders — tasks/{task}/maps.yml

Each task folder contains a `maps.yml` that tells the renderer what to do with that task's outputs. The simplest form is just a list of `yml_keys`:

```
# tasks/slope/maps.yml
layers:
  - slope
```

More options are available when you need aggregation, dependencies, or custom scripts:

Field	What it does
<code>layers:</code>	List of <code>yml_keys</code> this task contributes (must exist in <code>layers.yml</code>)
<code>aggregate_fun:</code>	How to aggregate raster pixels per cell — <code>mean</code> , <code>sum</code> , <code>median</code> , <code>min</code> , <code>max</code>
<code>aggregate_mode:</code>	Aggregation grid type — <code>hexbin</code> , <code>resample</code> , <code>h3</code> , or <code>none</code>
<code>aggregate_size:</code>	Target cell size in meters
<code>smoothing:</code>	Pre-smoothing before rendering — <code>method:</code> <code>gaussian</code> / <code>median</code> / <code>modal</code> , <code>window:</code> , <code>sigma:</code>
<code>depends:</code>	Other tasks whose layers must be rendered too (used by custom scripts)
<code>custom:</code>	List of R script paths (from repo root) for non-standard maps

Aggregation example — population rendered as summed hexbins:

```
# tasks/worldpop/maps.yml
layers:
  - population
aggregate_fun: sum
```

Dependencies and custom scripts — flood maps that need built-up and population overlays:

```
# tasks/fathom/maps.yml
layers:
  - fluvial
  - pluvial
```

```

- coastal
- combined_flooding
depends:
- worldpop
- wsf
- accessibility
custom:
- core/R/map-flooding.R

```

The `depends:` entries ensure that those tasks' standard layers are also rendered (so the custom script can read them). The `custom:` script then reads from `spatial_dir` and assembles whatever composite map it needs.

8.1.4 Adding a custom map script

Use a custom script when the standard pipeline — one raster, one style — isn't enough. Common cases: overlaying two data sources, combining flood layers with a built-up hatch, or rendering a temporal sequence.

Pattern:

```

# core/R/map-deforestation.R
forest <- fuzzy_read(spatial_dir, "forest_cover")
deforest <- fuzzy_read(spatial_dir, "deforestation")

p <- plot_static_layer(data = deforest, yaml_key = "deforest",
  plot_aoi = TRUE, plot_wards = !is.null(wards)) +
  geom_spatraster(data = forest, fill = "#8CC056", alpha = 0.5)

save_plot(p, filename = "forest_deforest.png", directory = styled_maps_dir)

```

Key functions available (loaded by `maps-static.R`):

- `fuzzy_read(dir, pattern)` — find and read a file by partial name or regex
- `plot_static_layer(data, yaml_key, ...)` — create a styled ggplot using `layers.yml` settings
- `save_plot(plot, filename, directory)` — save with consistent legend layout and dimensions

- `add_builtup_hatch(plot, underlay = FALSE)` — overlay a 2025 built-up area hatch pattern; set `underlay = TRUE` to place it beneath point layers

Hook the script into the task by adding it to `maps.yml`:

```
# tasks/forest/maps.yml
layers:
  - forest
  - deforest
custom:
  - core/R/map-deforestation.R
```

To render only this task's maps without re-rendering everything:

```
scan forest --render maps --scan-id 2026-04-namibia-windhoek
```

8.2 Part 2: Charts

8.2.1 How chart rendering works

Each task can have a `charts/` folder containing a Quarto document. Running:

```
scan --render charts {task} --scan-id {id}
```

renders `tasks/{task}/charts/index.qmd` into an HTML file. All chart files across tasks are then assembled into a single combined report by:

```
scan --render scan-calculations --scan-id {id}
```

The order of tasks in that report is controlled by `scan-calculations/sections.yml`.

8.2.2 File structure

A chart folder needs two files:

```
tasks/{task}/charts/  
  index.qmd      # the chart document  
  _quarto.yml    # Quarto project config - copy from any other task
```

The `_quarto.yml` is nearly identical across all tasks:

```
project:  
  type: default  
  render:  
    - index.qmd  
  
format:  
  html:  
    theme: [default, ../../../../source/pagedtable-fix.scss]
```

Copy this from an existing task and change nothing. The `pagedtable-fix.scss` file applies Bootstrap 5 table styling fixes automatically.

8.2.3 Anatomy of a chart document

Every `index.qmd` has the same four parts. Here is the elevation chart broken down piece by piece.

1. YAML frontmatter

```
---  
df-print: paged  
execute:  
  echo: false  
  warning: false  
  message: false  
---
```

`df-print: paged` enables `print_paged_df()` for interactive HTML tables. The `execute` options suppress code, warnings, and messages from appearing in the rendered output.

2. Section header

Elevation

Each chart section starts with a **##** header. This becomes the section title in the combined scan-calculations report, so keep it short and descriptive.

3. Setup chunk (hidden, runs once)

```
#| label: setup-elevation
#| include: false
if (file.exists("_quarto.yml")) here::i_am("tasks/elevation/charts/index.qmd")
if (!exists("aoi")) {
  USE_GCS <- FALSE
  source(here::here("core/R/setup.R"))
}
```

`here::i_am()` anchors the project root so `here()` resolves correctly when rendering from inside the `charts/` folder — without it, `here()` points to the charts folder instead of the city root. The `source(setup.R)` call loads all libraries and creates the shared objects listed in the table below. The `if (!exists("aoi"))` guard prevents re-running setup when rendering via scan-calculations, which sources this file into an already-loaded session.

4. Data-loading chunk (hidden)

```
#| label: data-loading-elevation
#| include: false
elevation <- tryCatch({
  read_csv(str_subset(list.files(tabular_dir, full = T), "elevation.csv"),
    col_types = "cd") %>%
  rename(Bin = 1, Count = 2) %>%
  filter(!is.na(Bin), Count > 0) %>%
  mutate(percent = Count / sum(Count),
    Bin = factor(Bin, levels = Bin))
}, error = function(e) { message("elevation not available: ", e$message); NULL })
```

Wrap all data loading in `tryCatch` — if the CSV doesn't exist yet, the variable is set to `NULL` and the chart section is skipped gracefully rather than crashing the render. Note the explicit `factor(..., levels = Bin)` call: factor levels are lost in CSV round-trips and must be restored after reading.

5. Chart chunk

```

#| label: elevation-chart
#| results: asis
if (is.null(elevation) || nrow(elevation) == 0) {
  print_text(paste_bold("No elevation data available"))
} else {
  elevation_plot <- ggdonut(elevation, "Bin", "percent",
    colors = c("#F2D7D3", "#DCBAC5", "#C59FB8", "#AF84A9", "#97679C"))

  print(elevation_plot)
  cat("\n\n")

  print_paged_df(elevation)
}

```

Always guard against empty data before plotting. Use plain `print()` for ggplots — `ggplot2` ≥ 4.0 removed `:::print.ggplot()`. The `cat("\n\n")` after `print()` is required to prevent subsequent text from rendering beside the plot. `print_paged_df()` renders a paginated HTML table.

8.2.4 Objects available in chart documents

After `source(setup.R)` runs, these objects are available in every chart:

Object	Contents
<code>tabular_dir</code>	Path to 02-process-output/tabular/ — read CSVs from here
<code>spatial_dir</code>	Path to 02-process-output/spatial/ — read shapefiles/rasters from here
<code>city_params</code>	Parsed <code>city_inputs.yml</code> as a list
<code>city</code>	City name, title case
<code>country</code>	Country name, title case
<code>aoi</code>	City boundary as a <code>SpatVector</code> (EPSG:4326)
<code>wards</code>	Wards boundary, or NULL if not present
<code>layer_params</code>	Parsed <code>source/layers.yml</code> — use for palettes/breaks in charts

8.2.5 Adding a new section to an existing chart

Open the task's `index.qmd` and add a new `##` section following the same pattern:

1. Load the CSV from `tabular_dir` in a hidden chunk with a `tryCatch` guard
 2. Add a `## Section Title` header
 3. Build the `ggplot` in a `results: asis` chunk
 4. Call `print(plot)`, then `cat("\n\n")`, then `print_paged_df(df)` if a table is also needed
-

8.2.6 Creating a chart for a new task

1. Create `tasks/{task}/charts/`
2. Copy `_quarto.yml` from any existing task (e.g. `tasks/solar/charts/_quarto.yml`)
3. Create `index.qmd` using the boilerplate structure above — adjust the `here::i_am()` path and the CSV filename

4. Render to check output:

```
scan --render charts {task} --scan-id {id}
```

5. To include this task in the combined scan-calculations report, add it to `scan-calculations/sections.yml` in the appropriate position

Part IV

Tasks & Data Layers

9 Tasks & Structure Overview

City Scan is organized around modular **tasks** — each responsible for computing a single data layer. This chapter explains how tasks are structured and how they interact.

9.1 What Is a Task?

Lorem ipsum dolor sit amet, consectetur adipiscing elit. A task is a self-contained Python module that ingests raw data, performs spatial analysis, and writes output files. Each task is independently runnable and cacheable.

```
tasks/  
  population.py  
  landuse.py  
  roads.py  
  flood_risk.py  
  ...
```

9.2 Task Anatomy

9.2.1 Inputs, Processing, Outputs

```
# Typical task structure  
class PopulationTask:  
    name = "population"  
    inputs = ["aoi", "worldpop"]  
    outputs = ["population_density.tif", "pop_stats.csv"]  
  
    def run(self, config):  
        aoi = self.load_aoi(config["aoi"])  
        data = self.fetch_worldpop(aoi, config["year"])  
        result = self.clip_and_resample(data, aoi)  
        self.write_outputs(result, config["output_dir"])
```

9.2.2 Task Metadata

Each task declares metadata used by the orchestrator:

Table 9.1: Task metadata fields

Field	Type	Description
<code>name</code>	<code>str</code>	Unique task identifier
<code>inputs</code>	<code>list[str]</code>	Required input data sources
<code>outputs</code>	<code>list[str]</code>	Files this task produces
<code>depends_on</code>	<code>list[str]</code>	Tasks that must run first

9.3 Task Categories

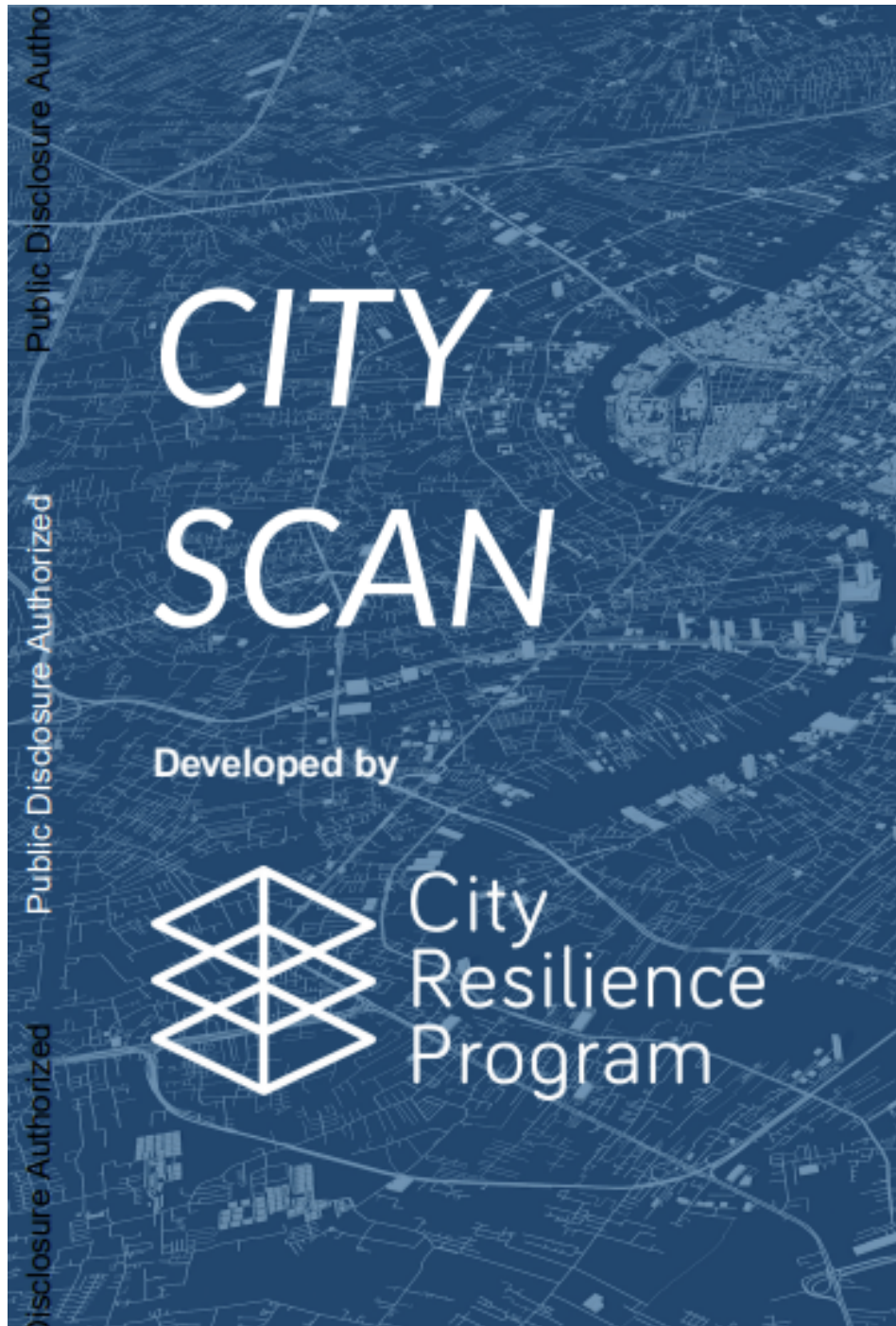


Figure 9.1: Task category overview

Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tasks fall into four broad categories:

- **Population** — WorldPop-based density and count estimates
- **Environment** — NDVI, NDWI, land use, and urban heat
- **Infrastructure** — roads, buildings, and accessibility
- **Risk** — flood, coastal erosion, and seismic exposure

9.4 Execution Model

Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. The orchestrator resolves task dependencies and runs tasks in topological order, parallelizing independent tasks where possible.

```
# Run all tasks
python run.py --config config.yml

# Run a specific task
python run.py --config config.yml --task population
```

Note

See [Data Layer References](#) for a complete list of tasks and their data sources.

10 Data Layer References

Complete reference for all data layers available in City Scan, including source datasets, resolution, and output descriptions.

10.1 Population

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Population layers are derived from WorldPop and LandScan datasets.

Table 10.1: Population data layers

Task ID	Description	Source	Resolution
population	Total population count per grid cell	WorldPop	100m
population_density	People per km ²	WorldPop	100m
pop_growth	Annual population growth rate	WorldPop (multi-year)	100m

10.2 Land Use & Environment

```
# Example: computing NDVI
def compute_ndvi(red_band, nir_band):
    return (nir_band - red_band) / (nir_band + red_band)
```

Table 10.2: Land use and environment data layers

Task ID	Description	Source	Resolution
landuse	Land use / land cover classification	Dynamic World (GEE)	10m

Task ID	Description	Source	Resolution
ndvi	Normalized Difference Vegetation Index	Sentinel-2	10m
ndwi	Normalized Difference Water Index	Sentinel-2	10m
urban_heat	Land surface temperature	Landsat 8/9	30m

10.3 Infrastructure

Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Infrastructure layers use OpenStreetMap as the primary source.

Table 10.3: Infrastructure data layers

Task ID	Description	Source	Resolution
roads	Road network density	OpenStreetMap	Vector
buildings	Building footprints & density	OpenStreetMap	Vector
accessibility	Travel time to services	OpenStreetMap + OTP	100m

10.4 Risk & Hazard

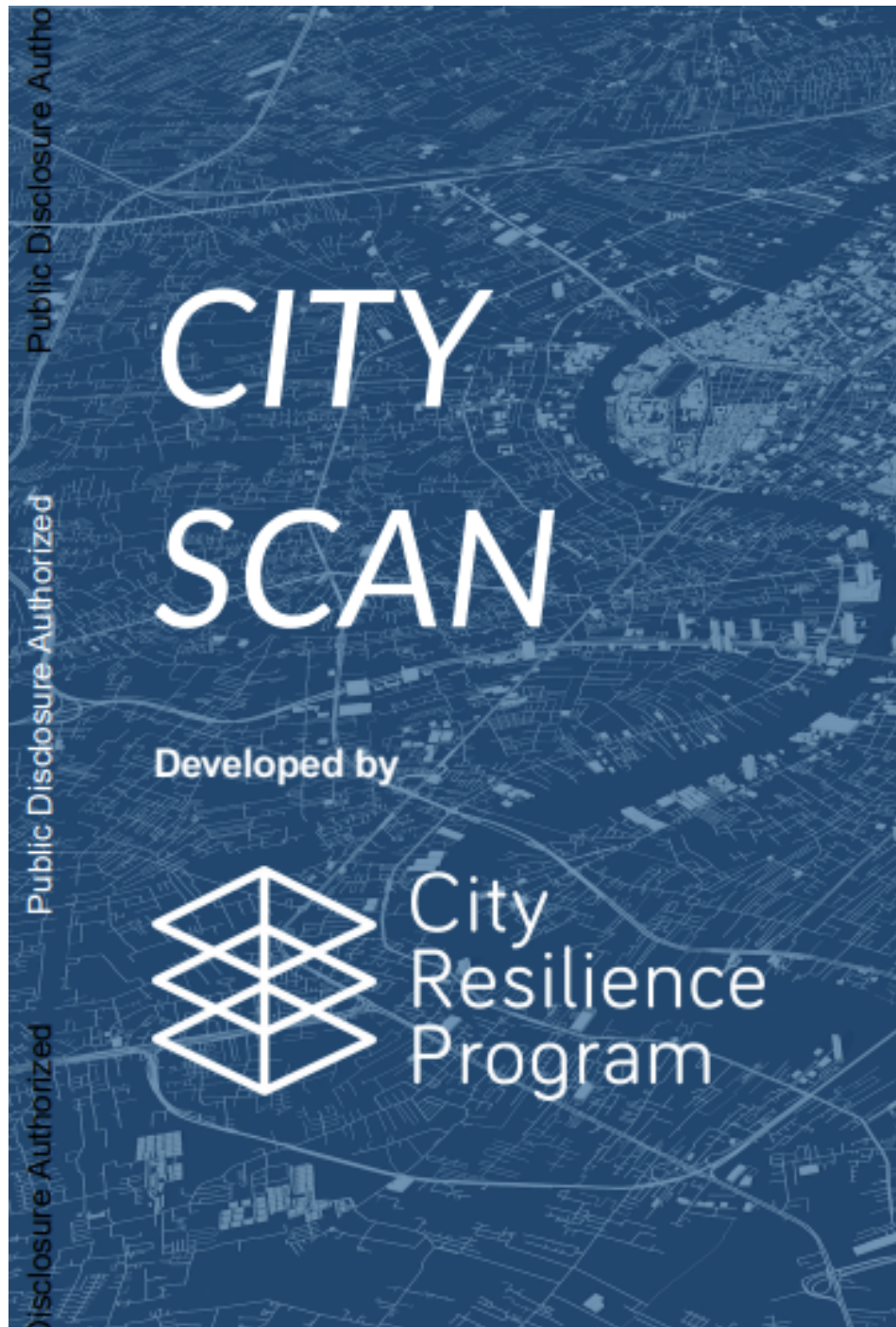


Figure 10.1: Risk layer visualization

Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.

Table 10.4: Risk and hazard data layers

Task ID	Description	Source	Resolution
flood_risk	100-year flood extent	Fathom / MERIT Hydro	90m
coastal_erosion	Shoreline change rate	CoastalDEM + Sentinel-1	30m
seismic	Peak ground acceleration	GSHAP	~10km

10.5 Adding a Custom Data Layer

```
# Register a new task
from cityscan.registry import register_task

@register_task
class MyCustomTask:
    name = "my_custom_layer"
    inputs = ["aoi"]
    outputs = ["my_custom_layer.tif"]

    def run(self, config):
        # implementation here
        pass
```

Tip

Custom tasks can be placed in the `tasks/custom/` directory and will be auto-discovered at runtime.

Part V

Understanding the Repo

11 Repository Structure

An orientation to the City Scan codebase — what lives where and why.

11.1 Top-level Layout

```
city-scan/
  cityscan/          # core Python package
  tasks/             # task modules
  inputs/            # user-supplied inputs (AOI, local data)
  outputs/           # generated scan outputs (git-ignored)
  scripts/           # utility scripts
  tests/             # test suite
  docs/              # this documentation
  pyproject.toml     # package metadata & dependencies
  run.py             # CLI entry point
  orchestrator.sh    # shell-based orchestrator (legacy)
```

11.2 Core Package: cityscan/

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

```
cityscan/
  __init__.py
  config.py          # config loading and validation
  registry.py        # task registration and discovery
  runner.py          # task execution and dependency resolution
  gee.py             # Google Earth Engine helpers
  io.py              # file I/O utilities
  report.py          # HTML report generation
```

11.2.1 Key Modules

Table 11.1: Core module responsibilities

Module	Responsibility
<code>config.py</code>	Load and validate <code>config.yml</code>
<code>registry.py</code>	Discover and register task classes
<code>runner.py</code>	Resolve dependencies, schedule, and run tasks
<code>gee.py</code>	Authenticate and interact with GEE

11.3 Tasks Directory: `tasks/`

```
tasks/  
  population.py  
  landuse.py  
  roads.py  
  flood_risk.py  
  coastal_erosion.py  
  custom/          # user-defined custom tasks
```

Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Each file in `tasks/` is a self-contained module that is auto-discovered by the registry at startup.

11.4 Inputs & Outputs

```
inputs/  
  AOI/              # place your GeoJSON boundary files here  
  
outputs/            # created automatically; git-ignored  
  <city_name>/  
    maps/  
    stats/  
    report.html
```

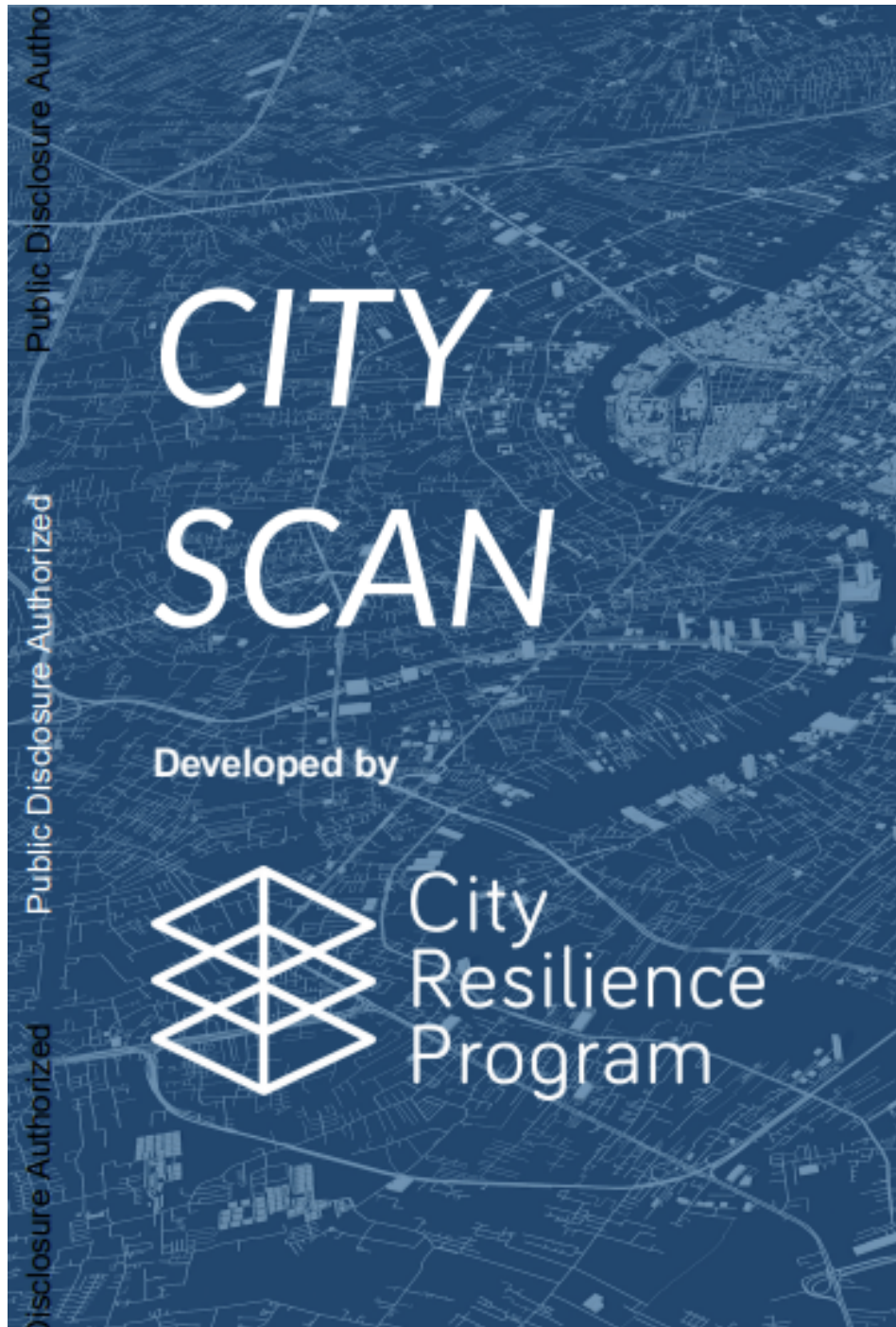


Figure 11.1: Repository structure diagram

11.5 Tests

```
pytest tests/          # run the full test suite
pytest tests/unit/      # unit tests only
pytest tests/integration/ # integration tests (requires GEE)
```

Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.

Note

For how tasks connect to each other, see [Task Dependency](#).

12 Task Dependency

City Scan tasks form a directed acyclic graph (DAG). Understanding dependencies helps you predict execution order and debug failures.

12.1 Dependency Graph

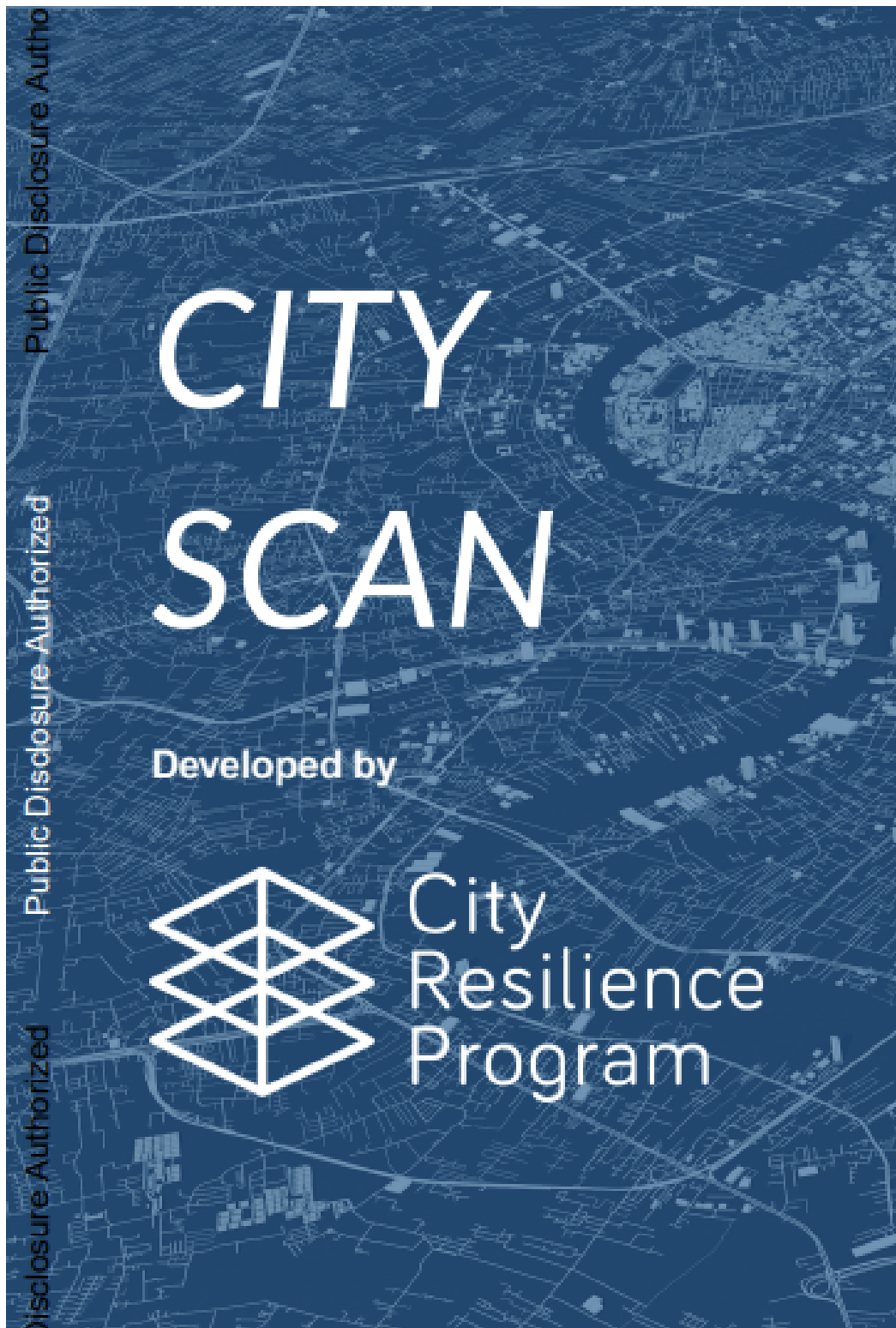


Figure 12.1: Task dependency DAG⁶⁸

Lorem ipsum dolor sit amet, consectetur adipiscing elit. The orchestrator builds this graph at startup by reading each task's `depends_on` declaration.

12.2 How Dependencies Are Declared

```
class AccessibilityTask:
    name = "accessibility"
    depends_on = ["roads", "population"] # must run after these

    def run(self, config):
        roads = self.load_output("roads")
        pop   = self.load_output("population")
        # ... compute accessibility
```

12.3 Execution Order Example

For the default task suite, the orchestrator produces the following topological order:

- | | |
|------------------|---------------------------------|
| 1. aoi_prep | |
| 2. population | (depends on: aoi_prep) |
| 3. landuse | (depends on: aoi_prep) |
| 4. roads | (depends on: aoi_prep) |
| 5. ndvi | (depends on: aoi_prep) |
| 6. flood_risk | (depends on: aoi_prep) |
| 7. accessibility | (depends on: roads, population) |
| 8. urban_heat | (depends on: landuse, ndvi) |
| 9. report | (depends on: all above) |

Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tasks at the same level can be parallelized.

12.4 Dependency Table

Table 12.1: Task dependency matrix

Task	Depends On
population	aoi_prep
roads	aoi_prep
landuse	aoi_prep
flood_risk	aoi_prep
accessibility	roads, population
urban_heat	landuse, ndvi
report	all tasks

12.5 Handling Failures

```
# Resume a scan after a failed task
python run.py --config config.yml --resume
```

Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. With `--resume`, the runner skips tasks whose output files already exist and only re-runs tasks downstream of the failure point.

12.6 Visualizing the DAG

```
from cityscan.runner import Runner

runner = Runner.from_config("config.yml")
runner.visualize_dag("dag.png") # saves a PNG of the dependency graph
```

Tip

Use `python run.py --dry-run --config config.yml` to print the resolved execution order without running any tasks.

Part VI

Case Studies & Galleries

13 Case Study: Standard Scan — Rabat, Morocco

This case study walks through a complete standard City Scan for Rabat, Morocco, demonstrating the end-to-end workflow and key findings.

13.1 City Context

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Rabat is the capital of Morocco, with a metropolitan population of approximately 1.9 million. The city faces urban growth pressures, coastal flood risk, and heat island effects.

Table 13.1: Rabat scan metadata

Attribute	Value
Country	Morocco
Population	~1.9 million
Area (AOI)	~117 km ²
Scan date	March 2024
Tasks run	14

13.2 AOI Definition

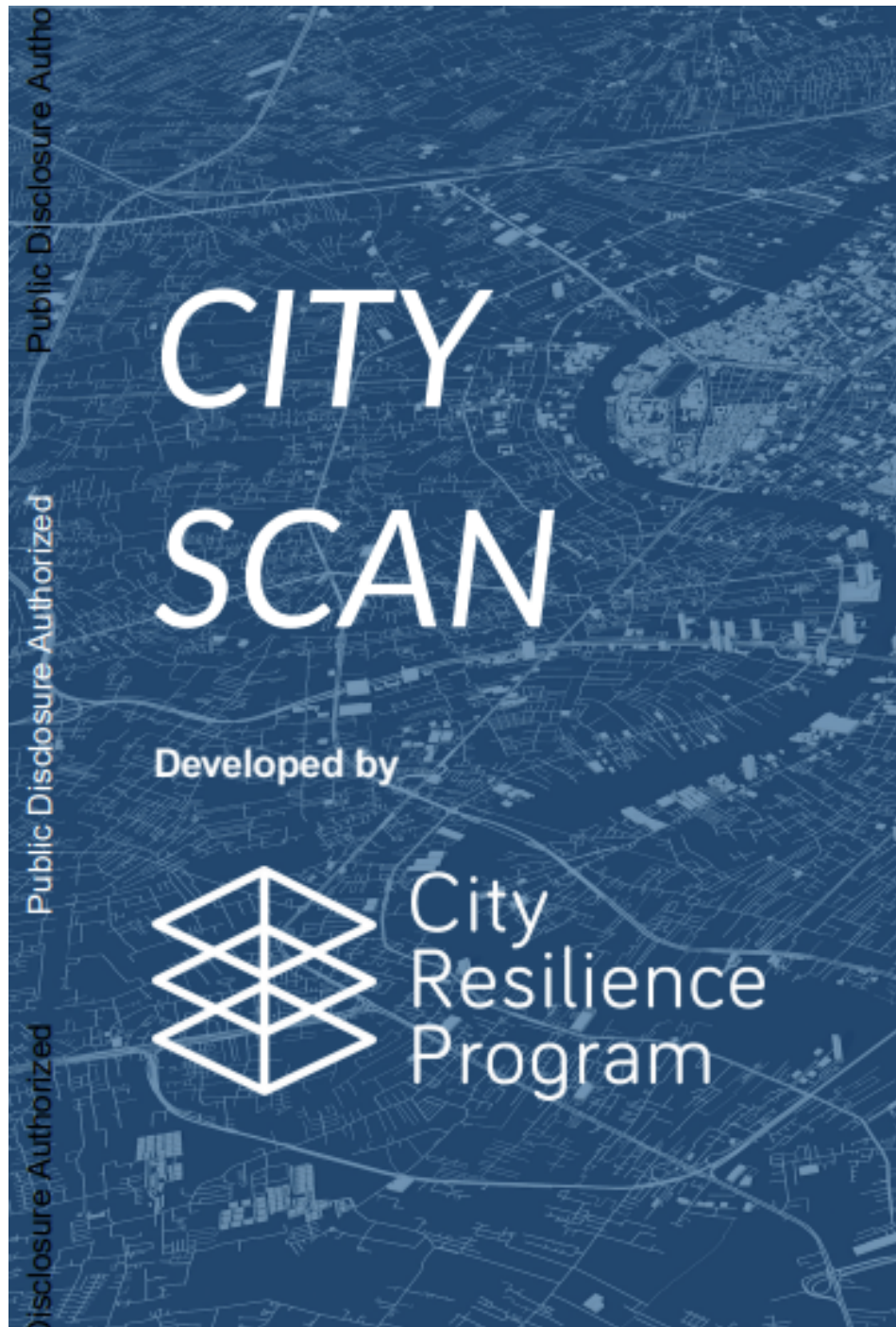


Figure 13.1: Rabat area of interest

Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. The AOI was defined using the official administrative boundary of the Rabat-Salé-Kénitra region, clipped to the primary urban core.

```
# AOI file used
inputs/AOI/rabat_urban_core.geojson
```

13.3 Running the Scan

```
python run.py --config configs/rabat.yml
```

```
# configs/rabat.yml
city: "Rabat"
country: "MA"
aoi: "inputs/AOI/rabat_urban_core.geojson"
output_dir: "outputs/rabat"
year: 2024
scan_type: standard
```

13.4 Key Findings

13.4.1 Population Distribution

Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Population is concentrated in the Agdal and Hassan neighborhoods, with density exceeding 15,000 people/km².

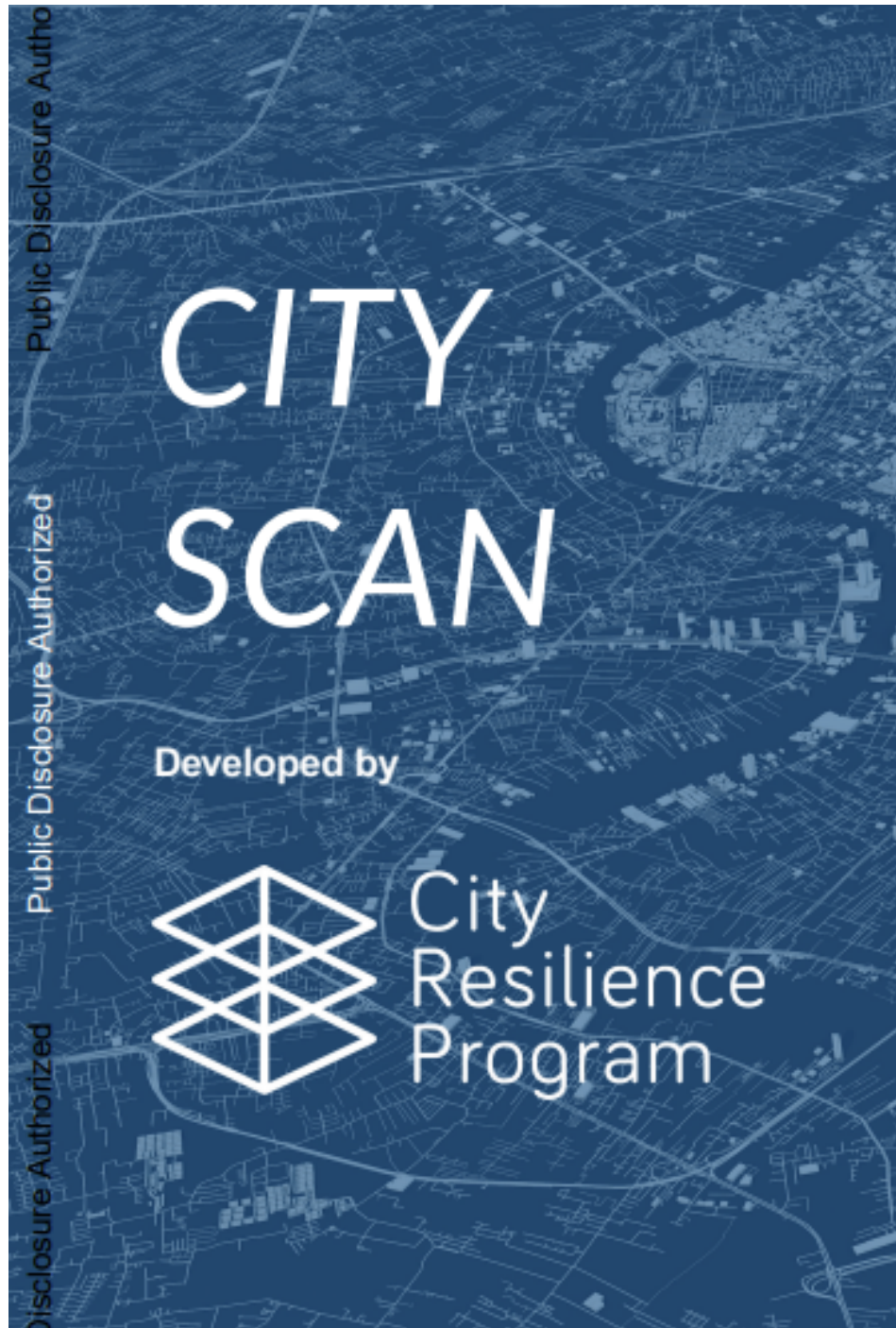


Figure 13.2: Rabat population density map

13.4.2 Flood Risk

Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Approximately 8% of the built environment falls within the 100-year flood extent, concentrated along the Bouregreg River corridor.

13.4.3 Land Use

```
# Summary statistics from outputs/rabat/stats/landuse_stats.csv
landuse_summary = {
  "Urban/Built-up": 62.3,
  "Vegetation": 18.7,
  "Bare soil": 11.4,
  "Water": 7.6,
}
```

13.5 Outputs

All scan outputs are available at `outputs/rabat/`. The interactive report can be viewed at `outputs/rabat/report.html`.

Note

For a multi-city batch scan case study, see [Case Study: Multiple Scans — Philippines](#).

14 Case Study: Multiple Scans — Philippines

This case study demonstrates running City Scan across multiple Philippine cities simultaneously using batch mode.

14.1 Project Context

Lorem ipsum dolor sit amet, consectetur adipiscing elit. The World Bank Philippines office requested City Scans for five secondary cities to support climate resilience planning. Batch mode was used to process all cities in a single run.

14.2 Cities Covered

Table 14.1: Philippine cities included in the batch scan

City	Region	Population	AOI Area
Cebu	Central Visayas	964,169	315 km ²
Davao	Davao Region	1,776,949	2,444 km ²
Iloilo	Western Visayas	457,626	78 km ²
Cagayan de Oro	Northern Mindanao	728,402	488 km ²
Zamboanga	Zamboanga Peninsula	977,234	1,483 km ²

14.3 Batch Configuration

```
# configs/philippines_batch.yml
scan_type: batch
output_base_dir: "outputs/philippines"
year: 2023

cities:
  - city: "Cebu"
```

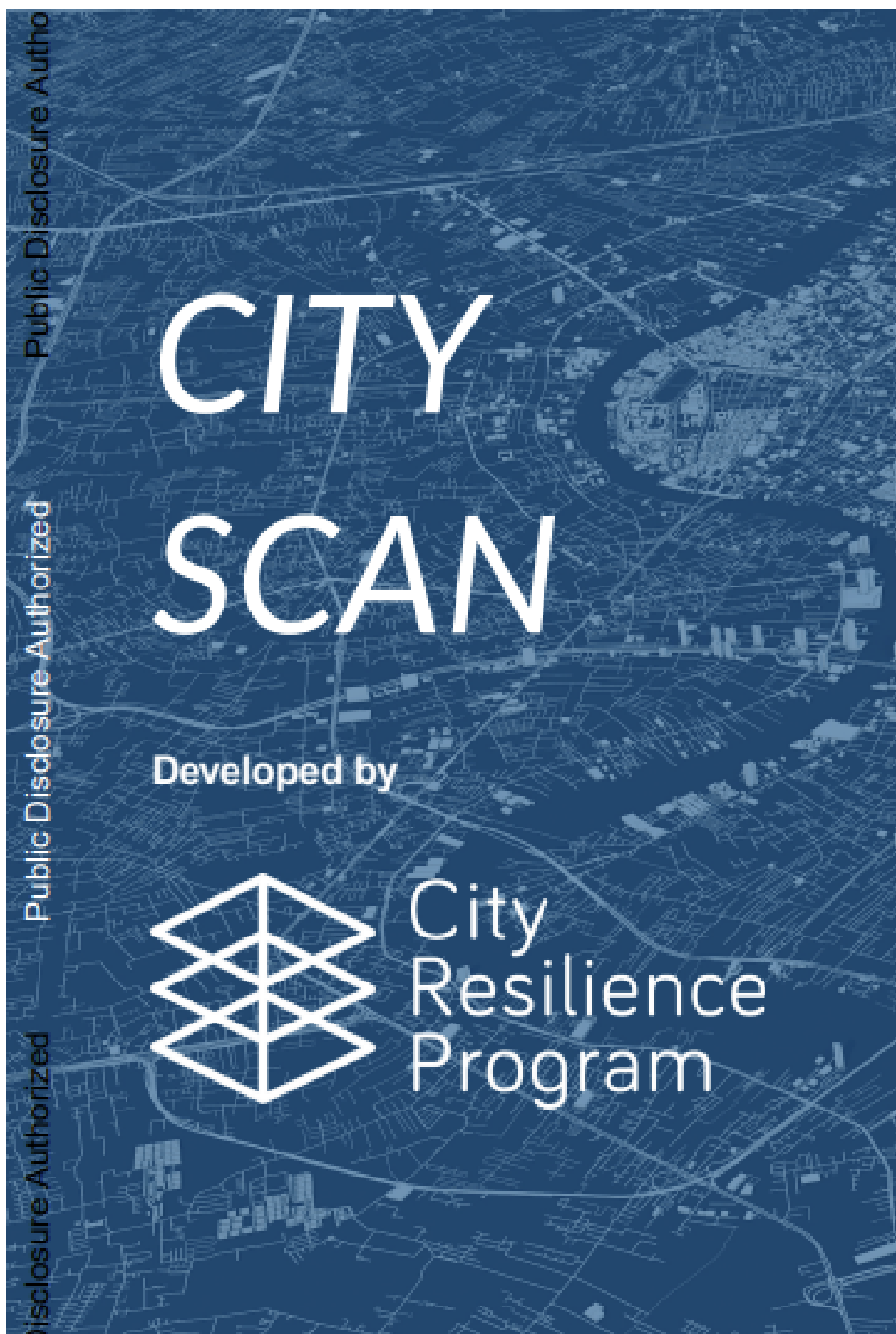
```
country: "PH"
aoi: "inputs/AOI/cebu.geojson"
- city: "Davao"
  country: "PH"
  aoi: "inputs/AOI/davao.geojson"
- city: "Iloilo"
  country: "PH"
  aoi: "inputs/AOI/iloilo.geojson"
- city: "Cagayan de Oro"
  country: "PH"
  aoi: "inputs/AOI/cdo.geojson"
- city: "Zamboanga"
  country: "PH"
  aoi: "inputs/AOI/zamboanga.geojson"
```

14.4 Running the Batch

```
python run.py --config configs/philippines_batch.yml --batch --workers 2
```

Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Two parallel workers were used to balance GEE API rate limits with execution speed. Total batch runtime was approximately 6 hours.

14.5 Key Findings Across Cities



81
Figure 14.1: Flood risk comparison across Philippine cities

Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.

14.5.1 Coastal Flood Risk

All five cities have significant coastal exposure. Zamboanga showed the highest percentage of built-up area within the 100-year flood extent at 23%.

14.5.2 Urban Heat Islands

```
# Comparative heat island intensity (°C above rural baseline)
heat_island = {
    "Cebu": 3.2,
    "Davao": 2.8,
    "Iloilo": 2.1,
    "Cagayan de Oro": 2.6,
    "Zamboanga": 3.5,
}
```

14.6 Lessons Learned

Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Key takeaways from this multi-city deployment:

- Pre-validate all AOI files before starting the batch to avoid mid-run failures.
- Use `--workers 2` as the safe default for GEE-heavy task suites.
- Monitor GEE quota usage via the [GEE Console](#) during long runs.

i Note

Configuration files for all five cities are available in `configs/examples/philippines/`.

Part VII

Collaborating

15 Git Guide

A practical Git reference for City Scan contributors, covering branching strategy, commit conventions, and common workflows.

15.1 Branching Strategy

Lorem ipsum dolor sit amet, consectetur adipiscing elit. City Scan uses a trunk-based development model with short-lived feature branches.

main	# stable, always deployable
feature/<name>	# new features or improvements
fix/<name>	# bug fixes
docs/<name>	# documentation-only changes

15.2 Cloning the Repository

```
git clone https://github.com/cityresilience/city-scan.git
cd city-scan
git checkout -b feature/my-new-feature
```

15.3 Commit Message Conventions

Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Follow the [Conventional Commits](#) format:

<type>(<scope>): <short description>

[optional body]

[optional footer]

Table 15.1: Conventional commit types

Type	When to Use
feat	New feature or data layer
fix	Bug fix
docs	Documentation changes only
refactor	Code restructuring, no behavior change
test	Adding or updating tests
chore	Maintenance (deps, CI, config)

15.3.1 Examples

```
git commit -m "feat(tasks): add seismic hazard layer"
git commit -m "fix(gee): handle expired credentials gracefully"
git commit -m "docs(setup): clarify GEE authentication steps"
```

15.4 Pull Request Workflow

```
# 1. Create a branch
git checkout -b fix/flood-risk-clipping

# 2. Make changes and commit
git add tasks/flood_risk.py
git commit -m "fix(tasks): correct AOI clipping in flood risk task"

# 3. Push and open a PR
git push -u origin fix/flood-risk-clipping
gh pr create --fill
```

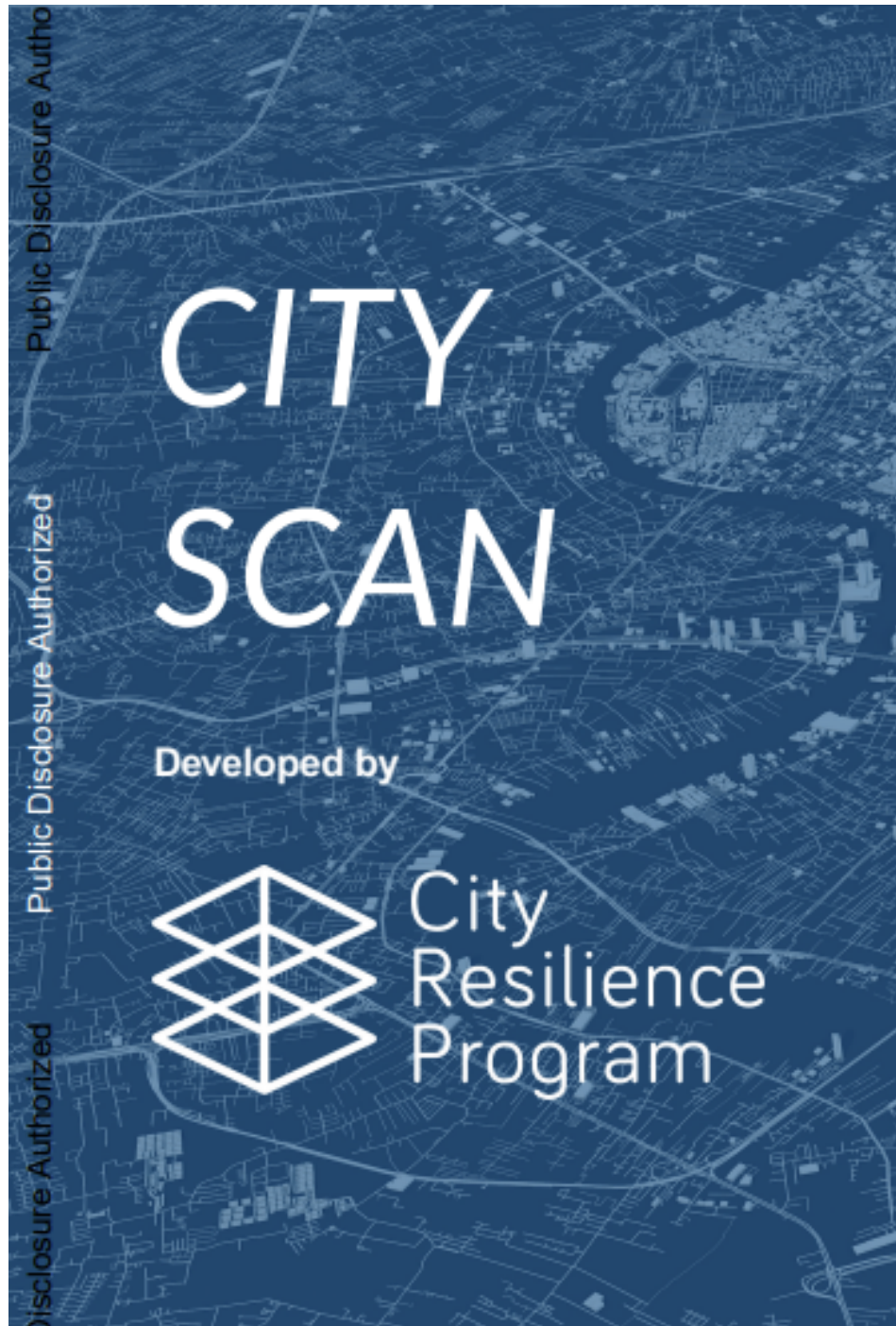


Figure 15.1: Pull request checklist on GitHub

15.5 Keeping Your Branch Up to Date

```
git fetch origin  
git rebase origin/main
```

Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.

Warning

Avoid `git merge main` into feature branches — use rebase to keep a clean linear history.

Note

For opening issues, see [Reporting Issues](#). For contributing code, see [Contributing](#).

16 Reporting Issues

How to report bugs, request features, and ask questions about City Scan.

16.1 Where to Report

Lorem ipsum dolor sit amet, consectetur adipiscing elit. All issues are tracked on the GitHub repository:

GitHub Issues: <https://github.com/cityresilience/city-scan/issues>

16.2 Before You Open an Issue

Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Before filing a new issue, please:

1. **Search existing issues** — your problem may already be reported or fixed.
2. **Check the documentation** — especially [Setting Up](#) for environment problems.
3. **Reproduce the issue** — confirm it happens on a clean install.

16.3 Bug Reports

Use the **Bug Report** template when something is broken. Include:

```
## Description
A clear description of what went wrong.

## Steps to Reproduce
1. Run `python run.py --config config.yml`
2. Observe error in `outputs/city/scan.log`

## Expected Behavior
The scan should complete successfully.
```



```
## Actual Behavior
Task `flood_risk` fails with `KeyError: 'return_period'`.

## Environment
- OS: macOS 14.4
- Python: 3.11.6
- City Scan version: 0.9.2
```

16.4 Feature Requests

Use the **Feature Request** template to propose new data layers, CLI options, or workflow improvements.

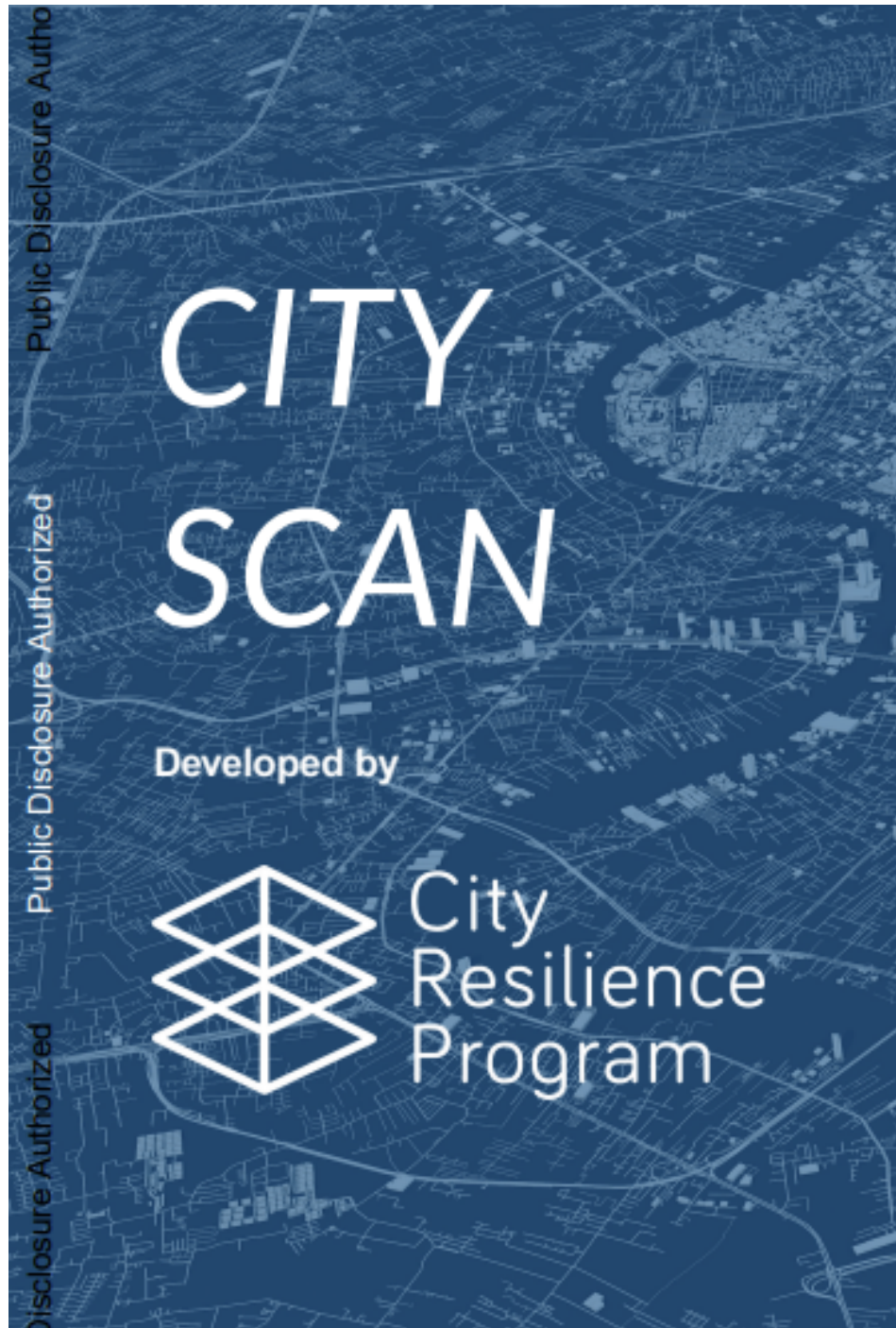


Figure 16.1: GitHub issue tracker showing a mix of bug reports and feature requests

16.5 Sharing Log Files

```
# Find your scan log
cat outputs/<city>/scan.log

# Or attach the whole log to the issue
```

Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Redact any sensitive paths or credentials before sharing logs.

16.6 Issue Labels

Table 16.1: GitHub issue labels

Label	Meaning
bug	Confirmed defect
enhancement	New feature or improvement
documentation	Docs gap or correction
good first issue	Suitable for new contributors
help wanted	Maintainers welcome external PRs

Tip

For quick questions, open a **Discussion** instead of an issue — discussions are better for open-ended conversations.

17 Contributing

We welcome contributions to City Scan — from new data layers to documentation improvements.

17.1 Types of Contributions

Lorem ipsum dolor sit amet, consectetur adipiscing elit:

- **New tasks** — add a data layer not currently in the suite
- **Bug fixes** — resolve issues in existing tasks or infrastructure
- **Documentation** — improve this cookbook
- **Tests** — expand test coverage
- **Performance** — optimize slow tasks or reduce GEE API calls

17.2 Development Setup

```
git clone https://github.com/cityresilience/city-scan.git
cd city-scan
python -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]"
pre-commit install
```

Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. The `[dev]` extras install linting, formatting, and test tools.

17.3 Code Style

City Scan uses [Ruff](#) for linting and formatting.

```
ruff check .          # lint
ruff format .         # format
```

Pre-commit hooks enforce these checks on every commit. To run manually:

```
pre-commit run --all-files
```

17.4 Writing a New Task

```
# tasks/my_new_layer.py
from cityscan.registry import register_task
from cityscan.base import BaseTask

@register_task
class MyNewLayerTask(BaseTask):
    name = "my_new_layer"
    description = "Computes lorem ipsum data layer."
    inputs = ["aoi"]
    outputs = ["my_new_layer.tif"]
    depends_on = []

    def run(self, config: dict) -> None:
        aoi = self.load_aoi(config["aoi"])
        result = self._compute(aoi)
        self.write_raster(result, "my_new_layer.tif")

    def _compute(self, aoi):
        # implementation here
        pass
```

Add a corresponding test in `tests/unit/test_my_new_layer.py`.

17.5 Pull Request Checklist

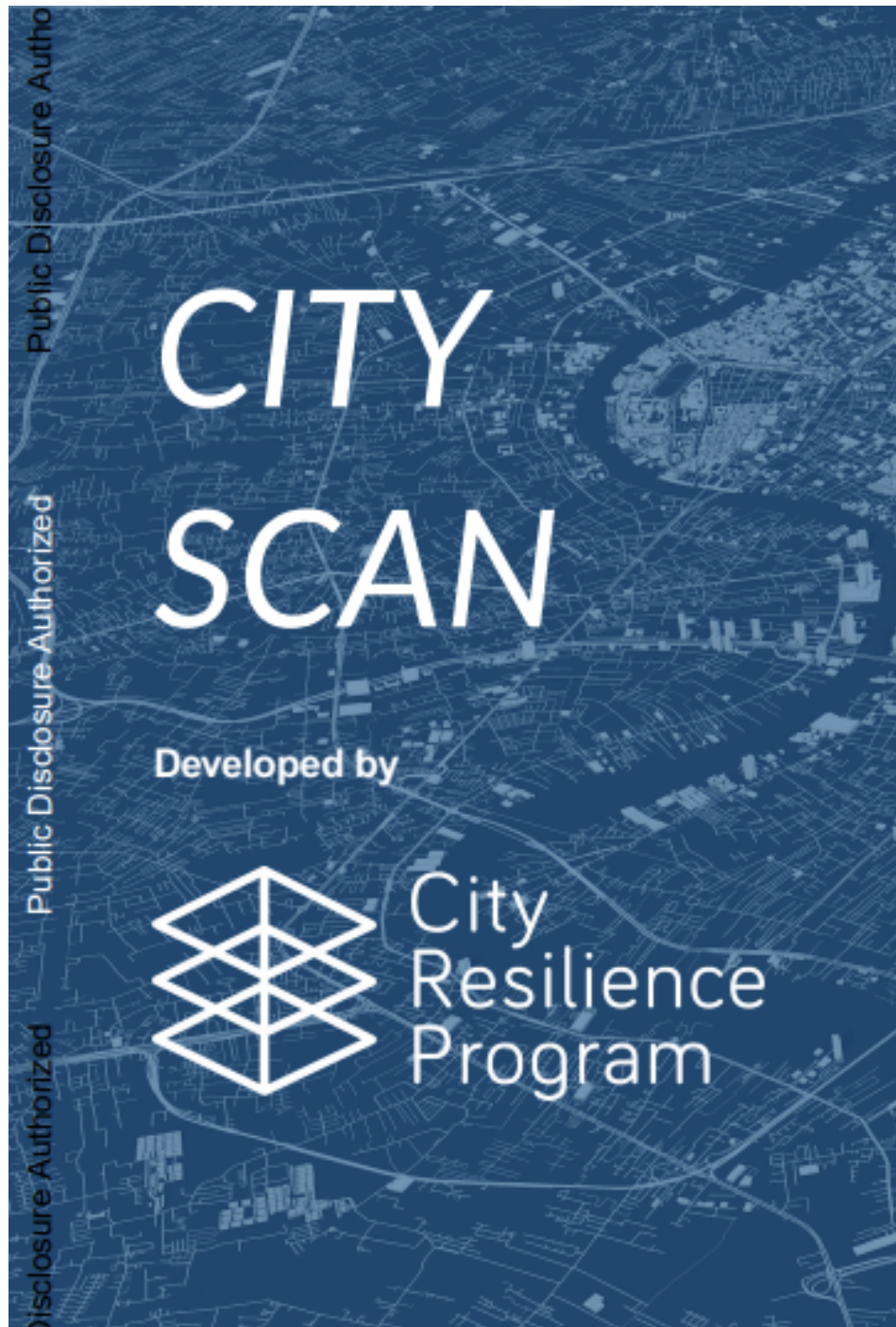


Figure 17.1: Pull request checklist

Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur.

- ☐ Tests pass: `pytest tests/`
- ☐ Linting passes: `ruff check .`
- ☐ New task is registered and documented in [Data Layer References](#)
- ☐ PR title follows Conventional Commits format
- ☐ No secrets or credentials committed

17.6 Review Process

Table 17.1: Pull request review stages

Stage	Who	Typical Duration
Automated checks (CI)	GitHub Actions	< 5 min
Code review	Core maintainer	1–3 days
Merge	Core maintainer	After approval

Note

See [Git Guide](#) for branching and commit conventions.

18 Sharing the Guide

How to share, publish, and distribute the City Scan Cookbook with your team and stakeholders.

18.1 Rendered Formats

Lorem ipsum dolor sit amet, consectetur adipiscing elit. The cookbook is built with [Quarto](#) and can be rendered to HTML, PDF, or both.

```
# Render HTML (default)
quarto render

# Render PDF
quarto render --to pdf

# Render both
quarto render --to html,pdf
```

Output is written to `_book/`.

18.2 Publishing to GitHub Pages

```
quarto publish gh-pages
```

Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. This command builds the book and pushes the rendered HTML to the `gh-pages` branch. The site is then accessible at:

<https://cityresilience.github.io/city-scan/>

18.3 Publishing to Quarto Pub

```
quarto publish quarto-pub
```

Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. You will be prompted to sign in to your Quarto Pub account on first use.

18.4 Sharing the PDF

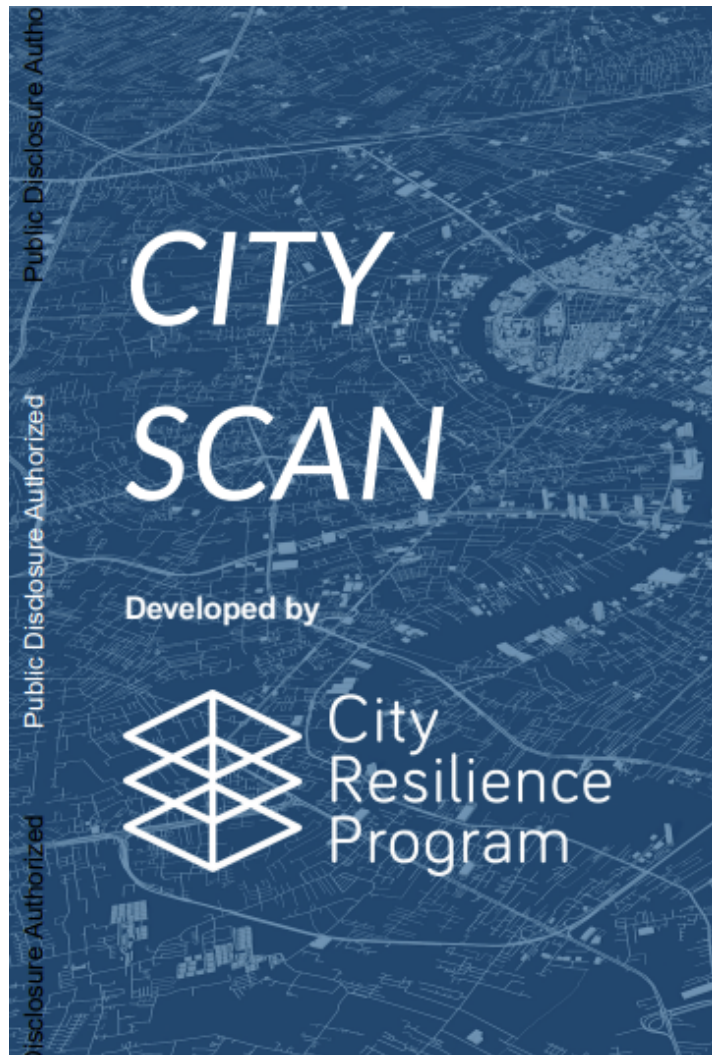


Figure 18.1: PDF export of the City Scan Cookbook

After rendering to PDF:

```
open _book/City-Scan-Cook-Book.pdf # macOS
```

Table 18.1: Guide distribution options

Distribution Method	Audience	Notes
GitHub Pages (HTML)	Technical users	Free, auto-updates on push
Quarto Pub (HTML)	General stakeholders	Free, public URL
PDF via email	Partners / clients	Snapshot in time
PDF on SharePoint	Internal World Bank	Behind org auth

18.5 Embedding in Other Sites

```
<!-- Embed a specific chapter as an iframe -->
<iframe
  src="https://cityresilience.github.io/city-scan/setting-up.html"
  width="100%"
  height="600px"
  frameborder="0">
</iframe>
```

18.6 Keeping the Guide Up to Date

```
# Set up GitHub Actions to auto-publish on every push to main
# .github/workflows/publish.yml
```

```
on:
  push:
    branches: [main]

jobs:
  publish:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

```
- uses: quarto-dev/quarto-actions/publish@v2
  with:
    target: gh-pages
```

 Tip

For feedback on the guide itself, open a **documentation** issue on GitHub — see [Reporting Issues](#).

19 Editing This Reference

A step-by-step guide for team members who want to update or extend this technical reference. Covers the editing workflow, branch conventions, and how to publish changes to GitHub Pages.

19.1 Prerequisites

- **Quarto** installed (quarto.org)
- **Git** installed and authenticated with GitHub
- Write access to the [cityresilience/city-scan](https://github.com/cityresilience/city-scan) repository

19.2 Repository Structure

Documentation lives in the `docs/` folder of the main `city-scan` repository. Each page is a Quarto Markdown file (`.qmd`):

```
city-scan/  
  docs/  
    _quarto.yml      # book structure and settings  
    index.qmd        # landing page  
    *.qmd            # one file per chapter  
    references.bib    # bibliography  
    image/           # shared images
```

The compiled website is published to the `gh-pages` branch and served at <https://cityresilience.github.io/city-scan/>.

19.3 Editing Workflow

19.3.1 1. Switch to new_docs

All documentation edits happen on `new_docs` — never commit directly to `main` or `gh-pages`.

```
git checkout new_docs
git pull origin new_docs      # always pull latest before editing
```

19.3.2 2. Edit .qmd files

Open the relevant `.qmd` file in `docs/` and edit it. Quarto Markdown is standard Markdown with optional callouts, cross-references, code blocks, and diagrams.

To preview changes live in your browser:

```
cd docs
quarto preview
```

This starts a local server (usually at `http://localhost:4848`) that hot-reloads as you save.

19.3.3 3. Add a new page (optional)

1. Create a new `.qmd` file in `docs/`, e.g. `docs/my-new-page.qmd`
2. Start the file with an H1 heading: `# My New Page`
3. Register it in `docs/_quarto.yml` under the appropriate `part:` section:

```
- part: "Collaborating"
  chapters:
    - editing-this-reference.qmd
    - my-new-page.qmd
```

19.3.4 4. Commit and push to new_docs

```
# Stage only docs source files - never stage docs/_book/
git add docs/my-new-page.qmd docs/_quarto.yml
git commit -m "docs: add page on <topic>"
git push origin new_docs
```

 Warning

Do **not** stage or commit anything inside `docs/_book/`. That folder contains compiled site output and is excluded by `docs/.gitignore`. Committing it to `new_docs` will bloat the repository with binary files.

19.4 Publishing Workflow

Once edits are committed to `new_docs`, publish to GitHub Pages from the `docs/` directory:

```
cd docs
quarto publish gh-pages
```

Quarto will:

1. Re-render only changed pages (`freeze: true` skips unchanged code outputs)
2. Commit the compiled site to the local `gh-pages` branch
3. Push `gh-pages` to `origin` — GitHub Pages auto-deploys within ~1 minute

When prompted:

Update site at <https://cityresilience.github.io/city-scan/>? (Y/n)

Type Y and press Enter.

 Note

First-time setup on a new machine: If you get an HTTP 400 error during the push step, run this once from the repo root, then re-publish:

```
git config http.postBuffer 524288000
```

This increases the git push buffer to 500 MB, which is needed for the initial upload of all site assets.

19.5 Branch Overview

Branch	Purpose	Who touches it
<code>main</code>	Stable code and configuration	Maintainers via PR
<code>new_docs</code>	Documentation source (<code>.qmd</code> files)	Editors directly
<code>gh-pages</code>	Compiled website — managed by Quarto	Never edit manually

19.6 Quick Reference

```
# Pull latest docs before editing
git checkout new_docs && git pull origin new_docs

# Start local preview
cd docs && quarto preview

# Commit documentation edits (from repo root)
git add docs/<changed-files>
git commit -m "docs: <short description>"
git push origin new_docs

# Publish to GitHub Pages (from docs/)
cd docs && quarto publish gh-pages
```

Note

For reporting content issues or suggesting structural changes, see [Reporting Issues](#). For contributing to the codebase itself, see [Contributing](#).

References